

Tensor Memory Layout

1. Overview

A PyTorch tensor is not merely a multidimensional array. It is a **view of a storage buffer** described by metadata: data type, device, sizes, strides, and a storage offset. The sizes describe the logical shape; the strides describe how far, in storage elements, to move when an index changes by one.

Memory layout matters because GPU kernels ultimately read linear memory. A logically correct tensor can still be slow if adjacent threads access widely separated addresses, if an operator must create a contiguous copy, or if a view has overlapping elements. Layout is used in every PyTorch workload: image tensors, attention matrices, embedding tables, batched matrix multiplication, and model parameters.

Interviewers ask about layout to test whether a candidate can connect high-level tensor operations to addresses, memory coalescing, copies, and kernel performance. A strong answer explains sizes, strides, offset, row-major order, and views rather than saying only that a tensor is “stored in GPU memory.”

2. Core Idea

Think of storage as a long street of numbered houses. A tensor's shape tells you the grid people imagine; its strides are the directions for finding the corresponding house.

For a two-dimensional tensor, the storage position of element (i, j) is:

```
storage_index = storage_offset + i * stride[0] + j * stride[1]
byte_address  = storage_base + storage_index * element_size
```

Example:

```
import torch

x = torch.tensor([[10, 11, 12],
                  [20, 21, 22]], device="cuda")

print(x.shape)          # torch.Size([2, 3])
print(x.stride())       # (3, 1)
print(x.storage_offset()) # 0
```

Step by step, $x[1, 2]$ maps to storage element $1*3 + 2*1 = 5$, which contains 22. PyTorch normally uses row-major, or C-contiguous, order: the last dimension changes fastest.

Now transpose it:

```
y = x.transpose(0, 1)
print(y.shape)      # [3, 2]
```

```
print(y.stride())      # (1, 3)
```

The data was not rearranged. Only the shape and strides changed. $y[2, 1]$ reaches $x[1, 2]$ through $2*1 + 1*3 = 5$. This cheap metadata operation is a **view**.

3. Important Subtopics

Storage, dtype, and element size

Storage is the one-dimensional allocation containing tensor data. The dtype determines how many bytes each logical storage element uses: typically 4 bytes for `float32`, 2 for `float16/bfloat16`, and 8 for `float64`.

Why it matters: strides are reported in **elements, not bytes**. A stride of 8 means 32 bytes for `float32` but 16 bytes for `float16`.

Example: a $[4, 8]$ `float32` contiguous tensor has stride $(8, 1)$ and 128 bytes of payload.

Interview angle: distinguish logical elements, storage elements, and byte addresses.

Shape and stride

`shape[d]` is the number of logical indices along dimension `d`. `stride[d]` is the storage-element jump caused by incrementing that index.

Why it matters: two tensors may have the same shape and values but different layouts. Kernels must honor strides unless they explicitly require a dense layout.

Example: a $[2, 3]$ tensor may have strides $(3, 1)$; its transpose has shape $[3, 2]$ and strides $(1, 3)$.

Interview angle: calculate the address of an indexed element.

Storage offset and sliced views

A view can begin inside another tensor's storage. `storage_offset()` records that starting position.

```
x = torch.arange(10, device="cuda")
y = x[3:8]
# y has size (5,), stride (1,), and storage_offset 3
```

Why it matters: `y.data_ptr()` refers to its first logical element, while the underlying storage can be larger. Holding a tiny view can keep a large allocation alive.

Interview angle: explain why slicing often does not copy and why view lifetime affects memory use.

Dense, contiguous, and memory format

The usual contiguous layout for shape $[N, C, H, W]$ has stride $[C*H*W, H*W, W, 1]$. PyTorch also supports the channels-last format, whose physical ordering is friendlier to some convolution kernels while the logical dimension order remains NCHW.

```
x = torch.empty(8, 64, 32, 32, device="cuda")
x_cl = x.to(memory_format=torch.channels_last)
print(x_cl.is_contiguous(memory_format=torch.channels_last)) # True
```

Why it matters: “contiguous” can be relative to a recognized memory format. Channels-last can improve Tensor Core/convolution paths, but only when the surrounding operators support it well.

Interview angle: do not equate channels-last with changing the logical shape to NHWC.

Views, aliases, and overlap

Views share storage. Modifying one alias may change another. Operations such as `transpose`, `narrow`, `select`, many slices, `view`, and `as_strided` can create views.

`expand` can produce zero strides: many logical positions refer to the same physical element. `as_strided` can even create partially overlapping views and must be used carefully.

Why it matters: aliases affect correctness, autograd version tracking, in-place operations, and memory lifetime.

Interview angle: a view is not necessarily contiguous, and not every reshape can be represented as a view.

Alignment and GPU coalescing

GPU global-memory hardware combines accesses from nearby threads into memory transactions when addresses are suitably aligned and adjacent. A favorable tensor layout makes the dimension traversed by consecutive threads have stride 1.

Example: in a simple elementwise kernel over the last dimension of a contiguous matrix, thread `t` reads element `t`, producing adjacent addresses. Iterating down a transposed column may give each thread a large stride.

Interview angle: layout does not change algorithmic complexity, but it can dramatically change effective bandwidth.

4. Real-World Example

Consider an image model whose activations have logical shape `[N, C, H, W]`. A convolution backend may prefer channels-last physical layout because channel values used together can be adjacent and because optimized Tensor Core kernels support it.

```
model = model.cuda().to(memory_format=torch.channels_last)
images = images.cuda().to(memory_format=torch.channels_last)
with torch.autocast("cuda", dtype=torch.float16):
    logits = model(images)
```

The optimization succeeds only if the whole relevant pipeline preserves the format. Repeatedly converting NCHW to channels-last and back adds copies that may exceed the kernel savings. Profiling should confirm the result.

Another practical case is attention. A transpose may cheaply produce the logical `[batch, heads, sequence, head_dim]` arrangement, but a following kernel may require a specific packed layout. Framework code often chooses between a stride-aware kernel and an explicit materialization based on the operator.

5. Diagrams / Mental Models

Logical tensor x, shape (2, 3), stride (3, 1)

	col 0	col 1	col 2
row 0	10	11	12
row 1	20	21	22

Linear storage:

index	0	1	2	3	4	5
value	10	11	12	20	21	22

Logical transpose y, shape (3, 2), stride (1, 3)

	col 0	col 1	
row 0	10	20	addresses 0, 3
row 1	11	21	addresses 1, 4
row 2	12	22	addresses 2, 5

Same storage; different index-to-address mapping.

Metadata	Question it answers
<code>device</code>	Where is the storage allocated?
<code>dtype</code>	How is each element represented?
<code>size()</code>	What indices are logically valid?
<code>stride()</code>	How does an index change map to storage?
<code>storage_offset()</code>	Where does this view begin in storage?
memory format	Which recognized dense ordering does it follow?

6. Common Interview Questions

- What describes a PyTorch tensor's memory layout?** Shape, strides, storage offset, dtype, device, and the referenced storage. Expected: give an index-to-address explanation. Mistake: mentioning shape alone.
- What is a stride?** The number of storage elements skipped when an index in one dimension increases by one. Expected: strides are in elements. Mistake: reporting them as bytes.
- How is `x[i, j]` located?** At `offset + i*stride[0] + j*stride[1]`, multiplied by element size for a byte address. Expected: include the storage offset. Mistake: assuming `i*number_of_columns+j` for

every layout.

4. **Does `transpose copy data`?** Normally no; it returns a view with permuted sizes and strides. Expected: a later operator may materialize a copy. Mistake: saying transpose always rearranges storage.
5. **What is row-major layout?** The last logical dimension is stored adjacently and has stride 1 in the standard contiguous case. Expected: give strides such as `(C, 1)` for `[R, C]`. Mistake: confusing row-major with GPU thread organization.
6. **Can tensors with the same shape have different strides?** Yes, through transposes, slices, memory formats, or custom strided views. Expected: same logical values do not imply same physical order. Mistake: assuming shape uniquely defines layout.
7. **What is `storage_offset`?** The index in the underlying storage at which a tensor view begins. Expected: connect it to slicing. Mistake: treating the slice's first element as storage index zero.
8. **What is a tensor view?** A tensor with its own metadata that shares underlying storage with another tensor. Expected: aliases and no data copy. Mistake: assuming all view operations work for every stride pattern.
9. **Why can layout affect GPU performance?** It controls address patterns, memory coalescing, cache behavior, vectorization, and whether copies are required. Expected: tie adjacent threads to adjacent addresses. Mistake: saying only "contiguous is always faster."
10. **What is channels-last in PyTorch?** A supported physical memory format optimized for some image operations while logical tensor dimensions remain `[N, C, H, W]`. Expected: mention format-aware contiguity. Mistake: claiming the public shape becomes NHWC.
11. **Can a small view retain a large allocation?** Yes. Storage remains alive while any view references it. Expected: mention cloning the needed data if releasing the large storage is important. Mistake: assuming view size equals allocated size.
12. **What danger does `as_strided` introduce?** It can create overlapping or out-of-bounds-like logical layouts if parameters are invalid; writes to overlapping views have ambiguous behavior. Expected: it is a low-level expert tool. Mistake: treating it as a safe general reshape.

7. Deep-Dive Questions

1. **How does a kernel support arbitrary layouts?** It converts a logical linear index into per-dimension indices, then uses sizes, strides, and offset to compute addresses. General indexing costs instructions; specialized contiguous paths can use simple pointer increments and vectorized loads.
2. **Why can a size-1 dimension have unusual strides yet still be contiguous?** Changing the index of that dimension never occurs, so its stride does not affect reachable element order. Contiguity checks can ignore some stride ambiguity for size-1 dimensions.
3. **What is a non-overlapping dense tensor?** Its elements occupy a compact region without different logical elements aliasing, although its dimension order may differ from standard row-major. Such layouts can often be permuted into a dense order without copying.
4. **Why is zero stride useful?** Broadcasting can represent a repeated dimension without allocating repeated values. Every index in that dimension reuses the same storage element. It is memory-efficient for reads but unsafe for independent in-place writes.
5. **How do views interact with autograd?** Autograd tracks view relationships and version counters so that in-place modification of saved data can be detected. Gradients through a view are transformed/accumulated back into the appropriate base locations.

8. Comparison Tables

Layout	Example stride for logical $[N, C, H, W]$	Strength	Risk
Standard contiguous	$(C * H * W, H * W, W, 1)$	Natural for many generic operations	Not always optimal for convolution kernels
Channels-last contiguous	$(H * W * C, 1, W * C, C)$ for ordinary nonzero sizes	Optimized convolution paths and channel-vector access	Format conversions or unsupported operators
Transposed view	Depends on permutation	Metadata-only transformation	Strided access or later copy
Expanded view	Contains zero strides	No allocation for repeated values	Aliasing makes in-place writes invalid/ambiguous
View	Copy		
Shares storage	Own storage allocation		
Usually cheap metadata change	Reads and writes element data		
Mutations may be visible through aliases	Independent after creation		
Can keep base storage alive	Lifetime independent of source storage		

9. Common Mistakes

- Treating a tensor as only $(\text{data pointer}, \text{shape})$ and forgetting strides and offset.
- Assuming every transpose, slice, or reshape copies data.
- Assuming every view is contiguous.
- Reading a stride value as bytes instead of elements.
- Believing channels-last changes the logical NCHW API shape.
- Claiming contiguous layout guarantees a fast kernel; compute, launch overhead, and kernel design also matter.
- Using `as_strided` without proving bounds and non-overlap requirements.
- Forgetting that an alias keeps the underlying storage alive.
- Performing in-place writes on expanded or overlapping views.

10. Edge Cases / Special Cases

- Zero-element tensors have valid metadata but no element address should be dereferenced.
- Size-1 dimensions can admit multiple equivalent stride values.
- An `expand` view commonly has stride 0 along an expanded dimension.
- Negative strides are not generally supported by ordinary PyTorch tensor views; operations such as `torch.flip` commonly materialize data rather than exposing a negative-stride view.
- Quantized, sparse, nested, and backend-specific tensors use layout models beyond ordinary strided dense tensors.
- A tensor can be contiguous in channels-last format but not in the default contiguous format.
- A slice with a step greater than one can be non-contiguous even when it is one-dimensional.
- Overlapping layouts may make in-place operations rejected or semantically unsafe.

11. How to Explain in Interview

"A PyTorch dense tensor is a strided view over one-dimensional storage. Its sizes define the logical index space, and its strides plus storage offset map an index to a storage location. Transposes and many slices change only metadata, so they are cheap views, but their access pattern may be less coalesced or require a later contiguous copy. On GPUs I examine the stride of the dimension traversed by neighboring threads and preserve a supported memory format across the pipeline."

12. Quick Revision Notes

- Address formula: $\text{base} + (\text{offset} + \sum \text{index}[d] * \text{stride}[d]) * \text{element_size}$.
- Standard contiguous layout has the last dimension at stride 1.
- Strides count elements, not bytes.
- A view shares storage; a clone owns copied storage.
- `transpose` usually swaps sizes and strides without moving data.
- Channels-last is physical layout metadata, not a change to logical NCHW shape.
- Zero stride represents broadcast reuse.
- Layout affects coalescing, cache use, vectorization, and copy requirements.
- Trap: "same shape" does not imply "same layout."

13. Practice Tasks

1. Create a `[2,3,4]` CUDA tensor and manually calculate its contiguous strides.
2. Apply `permute(2,0,1)` and predict shape, strides, and three element addresses before printing them.
3. Slice every second column and decide whether the result is contiguous.
4. Compare `data_ptr()`, `storage_offset()`, and storage sharing for a base tensor, slice, transpose, and clone.
5. Convert an image tensor to channels-last and inspect both default and channels-last contiguity checks.
6. Benchmark an elementwise reduction along stride-1 and high-stride dimensions using `torch.utils.benchmark` or CUDA events.
7. Use `torch.profiler` to find an implicit copy caused by a layout-sensitive operator.
8. Explain why modifying an expanded tensor in place is problematic.

14. Final Cheat Sheet

Item	Must remember
Core definition	A tensor is strided metadata over storage.
Address	$\text{offset} + \sum \text{index} * \text{stride}$, then multiply by element size.
Main performance issue	Neighboring GPU threads should preferably access nearby addresses.
Most asked	Stride calculation, transpose behavior, view vs copy, channels-last.
Key comparison	Logical shape describes indexing; physical layout describes placement.
Interview trap	A transpose can be zero-copy yet make the next kernel slower.
One-line answer	"Shape says what the tensor looks like; strides say where each element lives."

Contiguous vs Non-Contiguous Tensors

1. Overview

A tensor is **contiguous** when its elements follow a recognized dense memory order without gaps for its logical traversal. In the default format, the last dimension is adjacent and each preceding stride equals the product of the sizes after it, ignoring some size-1 ambiguities. A **non-contiguous** tensor has a different stride pattern, often because it is transposed, permuted, expanded, or sliced with a step.

This distinction matters because some operations can handle arbitrary strides, while others require or strongly prefer contiguous input. Calling `.contiguous()` materializes a correctly ordered copy only when needed. In real training and inference systems, accidental copies can consume memory bandwidth, increase peak memory, and add latency.

Interviewers ask this topic because it exposes whether a candidate understands views, **view** versus **reshape**, hidden copies, and why a logically cheap transpose can affect downstream GPU execution.

2. Core Idea

Imagine a book. A contiguous tensor is read page by page and line by line. A transposed view is an instruction saying, "read the first word of every line, then the second word of every line." The words have not moved, but the reading order now jumps around.

```
x = torch.arange(12, device="cuda").reshape(3, 4)
print(x.stride())           # (4, 1)
print(x.is_contiguous())    # True

t = x.t()
print(t.shape, t.stride())   # (4, 3), (1, 4)
print(t.is_contiguous())    # False

c = t.contiguous()
print(c.stride())           # (3, 1)
print(c.is_contiguous())    # True
```

Step by step:

1. `x` stores row elements adjacently.
2. `t()` swaps shape and strides but shares storage with `x`.
3. Traversing `t` in logical row-major order jumps through the old columns.
4. `t.contiguous()` allocates new storage and copies values in `t`'s logical order.
5. If called on an already contiguous tensor with the same requested format, `.contiguous()` commonly returns the tensor itself rather than copying.

3. Important Subtopics

The contiguity condition

For the default memory format, scan dimensions from last to first. Starting with expected stride 1, a non-size-1 dimension must have that stride; then multiply the expected stride by its size.

Example: shape `(2,3,4)` has canonical stride `(12,4,1)`. A stride `(12,1,3)` describes a different layout.

Interview angle: compute canonical strides and explain why size-1 dimensions are special.

Sources of non-contiguity

Common sources are `transpose`, `permute`, stepped slicing such as `x[:, ::2]`, `diagonal`, and `expand`. Basic narrowing can remain contiguous in some cases, but not all slices do.

Why it matters: “slice” does not imply either view or contiguity universally; inspect metadata.

Example: taking complete trailing rows may preserve dense order, while taking every second column creates gaps.

Interview angle: predict `is_contiguous()` for a sequence of operations.

`.contiguous()`

`.contiguous(memory_format=...)` returns a tensor laid out in the requested format. It is a no-op when the tensor already satisfies that format; otherwise it allocates and copies.

Why it matters: it can fix an operator precondition, but placing it casually in a hot loop creates recurring bandwidth and allocation costs.

Interview angle: `.contiguous()` is not an in-place rearrangement and may break storage aliasing.

`view` versus `reshape`

`view` requires that the requested shape be representable using compatible existing strides. `reshape` returns a view when possible but may silently copy otherwise.

```
t = torch.arange(12, device="cuda").reshape(3, 4).t()
# t.view(-1) may fail because the logical flattening crosses incompatible strides
r = t.reshape(-1) # succeeds; may allocate a contiguous copy
```

Why it matters: `reshape` is convenient but can hide a performance-relevant copy.

Interview angle: never promise that `reshape` is always zero-copy.

Operator support and TensorIterator

Many PyTorch elementwise operations use stride-aware iteration machinery and accept non-contiguous tensors directly. Matrix multiplication, convolutions, and backend library calls may select different algorithms, create internal copies, or impose layout requirements.

Why it matters: forcing contiguity before every operation may be slower than letting a stride-aware elementwise kernel operate directly.

Interview angle: profile the actual operator; there is no universal rule that explicit contiguity is best.

Autograd and gradient layout

Autograd differentiates through copies and views. A contiguous copy is not a gradient barrier. However, view aliases and in-place writes are tracked, and parameter gradient layouts may have performance implications for distributed reduction or optimizer access.

Example: `loss = x.t().contiguous().square().sum()` still propagates the correct gradient to `x`.

Interview angle: distinguish graph connectivity from storage sharing.

4. Real-World Example

An attention implementation changes `[batch, sequence, heads, head_dim]` into `[batch, heads, sequence, head_dim]` with `transpose`. That transpose is metadata-only. A fused attention kernel may accept the resulting strides directly, while another implementation may require the last two dimensions packed in a particular way.

```
q = projection(x).view(batch, seq, heads, head_dim)
q = q.transpose(1, 2) # often non-contiguous, usually no copy
scores = q @ k.transpose(-2, -1)
```

Blindly inserting `q = q.contiguous()` may add a large device-to-device copy each layer. Never inserting it may cause a backend to copy internally or choose a worse kernel. The practical process is:

1. Check the consumer's supported layouts.
2. Profile with realistic shapes.
3. Preserve the chosen layout through adjacent operations.
4. Materialize once at a deliberate boundary if multiple later operations benefit.

5. Diagrams / Mental Models

```
x: shape (2, 3), stride (3, 1), storage [a b c d e f]
```

```
logical traversal: a -> b -> c -> d -> e -> f
```

```
storage traversal: a -> b -> c -> d -> e -> f
                  contiguous
```

```
x.T: shape (3, 2), stride (1, 3), same storage
```

```
logical traversal: a -> d -> b -> e -> c -> f
```

```
storage positions: 0 -> 3 -> 1 -> 4 -> 2 -> 5
                  non-contiguous in default format
```

```
non-contiguous view
|
```

```
| consumer supports strides?
+-- yes --> direct strided kernel
|
+-- no --> materialize/copy --> dense kernel
```

6. Common Interview Questions

1. **What is a contiguous tensor?** A tensor whose stride pattern matches a recognized dense memory format for its shape. Expected: mention both default and format-aware contiguity. Mistake: defining it only as “stored in one block”; non-contiguous views also reference one block.
2. **How does a tensor become non-contiguous?** Commonly through transpose, permute, stepped slicing, diagonal extraction, or expansion. Expected: these are usually views. Mistake: saying non-contiguous means fragmented allocation.
3. **What does `.contiguous()` do?** It returns the tensor unchanged when already in the requested layout; otherwise it allocates and copies values into that layout. Expected: copy is conditional. Mistake: calling it an in-place operation.
4. **Why does `view` fail after a transpose?** The requested reshaping may not be expressible by merging/splitting the existing stride regions. Expected: `view` cannot arbitrarily reorder data. Mistake: blaming device type.
5. **How is `reshape` different from `view`?** `reshape` tries a view but may copy; `view` requires a compatible view layout and otherwise errors. Expected: hidden-copy risk. Mistake: saying they are always identical.
6. **Are non-contiguous tensors invalid on CUDA?** No. Many kernels support arbitrary strides. Expected: support and performance are operator-dependent. Mistake: claiming every CUDA op requires contiguity.
7. **Is contiguous input always faster?** No. A copy can cost more than direct strided access, and optimized kernels support several layouts. Expected: measure end-to-end. Mistake: optimizing only one kernel while ignoring conversion cost.
8. **Does `.contiguous()` detach autograd?** No. The copy operation remains in the computation graph. Expected: gradients flow back. Mistake: confusing new storage with a detached graph.
9. **Can a sliced tensor remain contiguous?** Yes, depending on which region is selected and the resulting strides; for example, selecting a contiguous block can remain dense. Expected: inspect shape/strides rather than using a slogan. Mistake: saying all slices are non-contiguous.
10. **What happens to aliases after `.contiguous()` copies?** The result owns different storage, so subsequent data mutations do not alias the original. Expected: graph relation can remain while storage aliasing ends. Mistake: confusing aliasing with gradient dependency.
11. **How do you detect layout?** Use `.stride()`, `.is_contiguous()`, and format-specific checks such as `.is_contiguous(memory_format=torch.channels_last)`. Expected: inspect the consumer too. Mistake: relying on shape.
12. **Why are hidden contiguous copies important in profiling?** They consume memory bandwidth, launch kernels, allocate temporary buffers, and increase peak memory. Expected: identify `aten::contiguous`, `aten::clone`, or copy operations in a trace. Mistake: measuring only the final math kernel.

7. Deep-Dive Questions

1. **When can dimensions be collapsed for a view?** Adjacent dimensions can be combined when their strides follow the dense relationship required by their sizes, subject to size-1 handling. A transpose

usually breaks the relationship across the swapped boundary.

2. **Why can an elementwise kernel handle a transpose without copying?** It maps each logical index using input/output strides. This adds address arithmetic and may reduce coalescing, but correctness does not require a packed layout.
3. **Could an explicit contiguous copy improve total runtime?** Yes, if a tensor is reused by several expensive consumers that benefit enough from packed access. The one-time copy can amortize. Measure copy plus all consumers.
4. **How does channels-last change the word contiguous?** PyTorch recognizes multiple dense memory formats. A tensor can be channels-last contiguous while `is_contiguous()` for the default format is false.
5. **Why are expanded tensors especially unusual?** They contain zero strides, so many logical elements alias one value. They are non-contiguous and read-efficient, but independent in-place updates cannot be represented safely.

8. Comparison Tables

Property	Contiguous tensor	Non-contiguous tensor
Logical correctness	Valid	Valid
Typical origin	Allocation, clone, compatible reshape	Transpose, permute, step slice, expand
Default stride	Canonical dense order	Different or zero-stride order
Generic elementwise ops	Supported	Usually supported
Library/kernel compatibility	Broad	Operator-dependent
Access/coalescing	Often simple	Can be less regular
Need for copy	None for matching layout	Only if consumer requires/benefits

API	Guarantees view?	May copy?	Typical use
<code>view</code>	Yes	No; errors instead	Reshape known-compatible layout
<code>reshape</code>	No	Yes	Convenient reshape when copy is acceptable
<code>flatten</code>	No	Yes	Flatten a dimension range
<code>contiguous</code>	No-op or copy	Yes if needed	Materialize requested dense format
<code>clone</code>	No	Always data copy	Independent storage while preserving logical strides when possible

9. Common Mistakes

- Defining non-contiguous as physically fragmented GPU memory.
- Calling `.contiguous()` after every `permute` without checking the consumer.

- Assuming `reshape` always returns a view.
- Assuming `.contiguous()` breaks gradient flow.
- Ignoring channels-last contiguity.
- Benchmarking an operation without including layout-conversion cost.
- Forgetting that direct strided kernels may still be bandwidth-efficient for some permutations.
- Using `view` after transpose and treating its error as a PyTorch bug.
- Assuming all slices are non-contiguous or all narrow operations are contiguous.

10. Edge Cases / Special Cases

- A scalar is trivially contiguous.
- Empty tensors and size-1 dimensions allow stride patterns that do not fit a naive equality test.
- A tensor can be contiguous in more than one format when dimensions are degenerate.
- `reshape` copy behavior can change when earlier layout transformations change; do not build correctness around whether it aliases.
- Expanded zero-stride tensors require cloning or another materialization before some writes.
- Some kernels accept non-contiguous inputs but return contiguous outputs; others preserve a suggested memory format.
- Advanced indexing commonly creates a copy, unlike many basic slices.
- An operation may internally materialize input even if Python code contains no `.contiguous()`.

11. How to Explain in Interview

"Contiguity is a property of shape and strides, not whether memory is one allocation. A transpose usually shares storage but changes strides, making the logical row-major traversal non-contiguous. Many PyTorch kernels accept this directly; `.contiguous()` should be used when a consumer requires a packed format or profiling shows the copy is amortized. `view` never copies and can fail, while `reshape` may silently copy."

12. Quick Revision Notes

- Default contiguous `[A,B,C]` stride: `(B*C, C, 1)`.
- Transpose/permute normally create non-contiguous views.
- `.contiguous()` is conditional copy, not in-place.
- `view`: view or error; `reshape`: view or copy.
- Many elementwise operations support arbitrary strides.
- Backend-heavy operations may require/prefer specific layouts.
- Include copy and allocation costs in benchmarks.
- New storage does not imply detached autograd.
- Trap: non-contiguous does not mean invalid or fragmented.

13. Practice Tasks

1. Predict `shape`, `stride`, and contiguity after `reshape`, `transpose`, `narrow`, and stepped slicing.
2. Make a transposed tensor; test `view(-1)` and `reshape(-1)` and inspect whether storage is shared.
3. Time `x + 1` on contiguous and transposed views of realistic size.
4. Time one explicit `.contiguous()` followed by ten operations versus ten direct strided operations.
5. Profile an attention block and locate any `contiguous`, `clone`, or copy kernels.
6. Test default and channels-last contiguity for a 4-D tensor.

7. Verify that gradients pass through `.contiguous()`.
8. Create an expanded tensor, inspect its zero stride, and explain why an in-place update is rejected or unsafe.

14. Final Cheat Sheet

Item	Must remember
Core definition	Contiguity means strides match a recognized dense format.
Main cause	Metadata-only views such as transpose and stepped slice.
Conversion	<code>.contiguous()</code> returns self or makes a copy.
Most asked	<code>view</code> vs <code>reshape</code> , transpose, hidden copies, channels-last.
Common comparison	Stride-aware direct execution vs materialize-then-execute.
Interview trap	A contiguous copy can make one kernel faster but the pipeline slower.
One-line answer	"Non-contiguous is a valid strided view; materialize it only for a reason."

Broadcasting

1. Overview

Broadcasting lets PyTorch apply an elementwise operation to tensors of different but compatible shapes without conceptually writing manual replication loops. Dimensions are aligned from the right. Two aligned dimensions are compatible when their sizes are equal, one of them is 1, or one dimension is absent.

Broadcasting matters because it expresses bias addition, normalization, masking, scaling, and batched arithmetic compactly. PyTorch can represent broadcasted reads using zero strides, avoiding a materialized repeated tensor. This saves memory capacity and bandwidth.

Interviewers ask about broadcasting to test shape reasoning, implicit expansion, gradient reduction, in-place restrictions, and the difference between a view-like expansion and an actual copy.

2. Core Idea

Imagine placing a transparent stencil over several sheets. A row-shaped stencil can be reused over every row without printing a separate copy. Logically, values repeat; physically, PyTorch can reread the same value.

```
x = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]], device="cuda") # [2, 3]
b = torch.tensor([10., 20., 30.], device="cuda") # [3]
y = x + b                                     # [2, 3]
```

Shape reasoning, step by step:

```

x: 2 x 3
b:      3      <- align dimensions from the right
-----
y: 2 x 3

```

The missing leading dimension of **b** behaves like size 1, then expands to 2. A kernel can calculate $y[i,j] = x[i,j] + b[j]$; no $[2,3]$ copy of **b** is required.

General compatibility algorithm:

1. Align shapes at the trailing dimension.
2. For each aligned position, accept equal sizes.
3. If one size is 1, expand it to the other size.
4. A missing leading dimension behaves like 1.
5. Otherwise the operation raises a shape mismatch error.

3. Important Subtopics

Broadcasting rules

For shapes $[8,1,6,1]$ and $[7,1,5]$, right alignment gives:

```

8 1 6 1
7 1 5
8 7 6 5   result

```

Why it matters: compare from the right, not from the left.

Interview angle: derive the output shape and identify the incompatible dimension when rules fail.

expand and zero strides

expand returns a view-like tensor in which a size-1 dimension can be presented as a larger dimension. The stride in that expanded dimension is usually zero.

```

b = torch.arange(3, device="cuda").view(1, 3) # stride (3, 1)
e = b.expand(4, 3)                            # stride (0, 1)

```

Why it matters: $e[i,j]$ reads $b[0,j]$ for every i , using no repeated storage.

Interview angle: explain why in-place writes on expanded views are unsafe.

repeat versus **expand**

repeat physically copies values in a tiled pattern. **expand** changes metadata and is possible only along dimensions whose original size is 1 (or newly introduced singleton dimensions).

Why it matters: **repeat** consumes memory but creates independent physical positions; **expand** is cheap but aliases.

Example: a `[1, 1024]` tensor expanded to `[4096, 1024]` still stores 1024 values, whereas repeating stores over four million.

Interview angle: choose **expand** for read-only broadcasting and **repeat** only when real replication is semantically/performance necessary.

Broadcasting in kernels

Elementwise iteration logic maps the output index to each input. For a broadcast dimension, the input address does not change. GPU kernels may specialize/coalesce dimensions to reduce indexing overhead.

Why it matters: broadcasting avoids reading a huge replicated operand from memory, though the same source value may still be loaded multiple times or served by cache/register paths.

Interview angle: "no allocation" does not mean "zero work." Output elements still must be computed and written.

Autograd through broadcasting

If a value was broadcast over several output positions, its gradient is the **sum** of contributions from all those positions.

```
x = torch.ones(2, 3, device="cuda", requires_grad=True)
b = torch.ones(3, device="cuda", requires_grad=True)
(x + b).sum().backward()
print(b.grad) # tensor([2., 2., 2.], device='cuda:0')
```

Why it matters: the backward pass performs a reduction over expanded dimensions.

Interview angle: candidates often incorrectly say the gradient is merely copied back.

In-place broadcasting rules

An in-place operation cannot change the shape of its destination. The other operand may broadcast to the destination, but the destination itself cannot expand.

```
x = torch.zeros(2, 3, device="cuda")
b = torch.ones(3, device="cuda")
x.add_(b) # valid: b broadcasts to x

a = torch.zeros(1, 3, device="cuda")
# a.add_(x) # invalid: destination a would need shape [2,3]
```

Why it matters: storage for the larger result does not exist in the smaller destination.

Interview angle: distinguish out-of-place result shape from in-place destination constraints.

4. Real-World Example

Layer normalization for activations x of shape `[batch, sequence, hidden]` uses learned `weight` and `bias` of shape `[hidden]`:

```
normalized = (x - mean) * inv_std
y = normalized * weight + bias
```

`weight` and `bias` broadcast across both batch and sequence. Materializing each as `[batch, sequence, hidden]` would waste memory. A fused layer-normalization kernel can load the relevant `weight[h]` and `bias[h]` while processing `x[b,s,h]`.

Another common case is attention masking:

```
# scores: [batch, heads, query_len, key_len]
# mask:   [batch, 1,      1,      key_len]
scores = scores.masked_fill(~mask, float("-inf"))
```

The mask broadcasts over heads and queries. Getting a singleton dimension wrong can silently produce a valid but unintended output shape, making shape assertions valuable.

5. Diagrams / Mental Models

```
b = [10 20 30]                physical values: 3

logical broadcast view:
[10 20 30]  -- all rows point to the same storage
[10 20 30]
[10 20 30]
[10 20 30]

stride = (0, 1)
         ^
         moving to next row advances zero storage elements
```

Aligned dimensions	Compatible?	Result size
5 and 5	Yes	5
1 and 7	Yes	7
missing and 4	Yes	4
0 and 1	Yes	0

Aligned dimensions	Compatible?	Result size
3 and 4	No	Error

6. Common Interview Questions

- What is broadcasting?** It is elementwise shape expansion under trailing-dimension compatibility rules, commonly without materializing repeated input data. Expected: equal, singleton, or missing dimensions. Mistake: saying any smaller tensor can broadcast.
- Are shapes `[3,1,5]` and `[1,4,5]` compatible?** Yes; the result is `[3,4,5]`. Expected: align every dimension. Mistake: comparing only total element count.
- Are `[2,3]` and `[3,2]` compatible?** No; from the right, 3 conflicts with 2 and 2 conflicts with 3. Expected: right alignment. Mistake: assuming transpose-like behavior.
- Does broadcasting copy data?** The operation usually handles it through strides/indexing, and `expand` uses zero strides. The output still allocates if the operation is out of place. Expected: separate logical input expansion from output allocation. Mistake: saying broadcasting allocates nothing at all.
- What is a zero stride?** Incrementing that logical dimension does not move the storage address, so values are reused. Expected: connect to expanded views. Mistake: interpreting zero stride as an empty dimension.
- How do gradients work for broadcast operands?** Contributions are summed over the dimensions that were broadcast. Expected: `sum_to_size`-style reduction. Mistake: selecting one contribution.
- What is the difference between `expand` and `repeat`?** `expand` aliases through zero strides without copying and only grows singleton dimensions; `repeat` tiles actual data. Expected: mention aliasing, storage, and singleton constraints. Mistake: saying both are views.
- Why are in-place writes to expanded tensors dangerous?** Several logical elements refer to one storage location, so separate updates collide or have undefined/unsupported meaning. Expected: clone/materialize before independent writes. Mistake: assuming each expanded value owns memory.
- Can an in-place binary operation broadcast?** The source can broadcast to the destination, but the destination's shape cannot grow. Expected: storage constraint. Mistake: applying out-of-place shape rules without the destination restriction.
- Why is broadcasting useful on GPUs?** It reduces memory footprint and avoids materializing repeated operands, often reducing memory traffic. Expected: output work still scales with output size. Mistake: claiming it makes the operation constant-time.
- How do you add a channel bias to NCHW data?** Reshape bias `[C]` to `[1,C,1,1]`, then add. Expected: place singleton dimensions explicitly. Mistake: adding `[C]` directly, which aligns it with `W`.
- Can broadcasting cause a performance or correctness bug?** Yes: an unintended large result can allocate huge memory, and valid-but-wrong alignment can silently compute the wrong semantics. Expected: assert shapes and inspect profiler/memory. Mistake: assuming a lack of runtime error proves intent.

7. Deep-Dive Questions

- What exactly happens in backward for $y = x + b$ where $b: [H]$ and $x: [B, S, H]$?** dL/dx has `[B, S, H]`; $dL/db[h]$ sums $dL/dy[b, s, h]$ over `b` and `s`. This reduction may require its own kernel unless fused.
- Why might broadcast access be efficient even with repeated loads?** The reused operand can be cached, and a kernel can load a scalar/vector once per thread block or keep it in registers. Efficiency

depends on access pattern and kernel implementation.

3. **What does `torch.broadcast_tensors` return?** Broadcasted views of its inputs with a common shape, commonly using zero strides; callers must beware in-place writes.
4. **Can size zero broadcast?** A dimension of size 0 is compatible with size 1 and yields size 0; it is not compatible with an unrelated size such as 2. Empty outputs contain no computed elements.
5. **How can broadcasting accidentally create an outer product?** Subtracting `[N, 1]` and `[M]` produces `[N, M]`. If the intended shapes were both `[N]`, the extra singleton dimension can turn linear work into quadratic work and memory.

8. Comparison Tables

Property	<code>expand</code>	<code>repeat</code>
Data copy	No	Yes
Storage growth	No	Yes
Allowed dimensions	Singleton/new leading dimensions	Any tiled pattern supported by repeat factors
Strides	May contain 0	Normally dense repeated storage
Independent in-place writes	No for aliased positions	Yes
Typical use	Read-only broadcast operand	Actual repeated data required

Broadcasting	Concatenation
Reuses values across compatible dimensions	Joins tensors along one dimension
Output dimensions usually take per-axis maximum	Concatenated dimension sizes are summed
Used by elementwise arithmetic	Used to assemble a larger tensor
Inputs need broadcast compatibility	Non-concatenated dimensions must match

9. Common Mistakes

- Aligning shapes from the left instead of the right.
- Assuming equal element counts imply broadcast compatibility.
- Using `[C]` as an NCHW channel bias without reshaping to `[1, C, 1, 1]`.
- Saying `expand` allocates repeated storage.
- Saying broadcasting creates no output allocation.
- Performing in-place writes on a zero-stride expanded view.
- Forgetting that backward sums over broadcast dimensions.
- Accidentally creating an outer product and a huge intermediate.
- Using `repeat` where implicit broadcasting or `expand` is sufficient.
- Assuming broadcasting is free; address calculation, arithmetic, output writes, and backward reductions remain.

10. Edge Cases / Special Cases

- Scalars (zero-dimensional tensors) broadcast with any shape.
- Missing leading dimensions are treated as singleton dimensions for compatibility.
- A size-0 dimension combined with size 1 produces size 0; empty tensors deserve explicit tests.
- Broadcasting does not use total element count to decide compatibility.
- `expand(-1, ...)` can preserve an existing dimension size, but a new leading dimension cannot use `-1` because there is no existing size to preserve.
- Expanded views can be non-contiguous even though reading them is valid.
- Named tensors, sparse tensors, and backend-specific operators can add constraints beyond ordinary dense broadcasting.
- Mixed dtypes also invoke type-promotion rules; broadcasting and dtype promotion are separate decisions.

11. How to Explain in Interview

"PyTorch broadcasting aligns shapes from the trailing dimension. Each pair must match or one must be 1; missing leading dimensions act like 1. The result takes the larger size. PyTorch can implement singleton expansion with a zero stride, so values are reused without materializing copies. In backward, gradients for a broadcast input are summed over the expanded axes, and in-place operations cannot expand their destination."

12. Quick Revision Notes

- Align dimensions from the right.
- Compatible: equal, one is 1, or one is missing.
- Result size is the non-1 size, with empty-dimension rules included.
- `expand` uses view-like zero strides; `repeat` copies.
- Broadcast output still contains all result elements.
- Gradient to a broadcast operand reduces by summation.
- In-place destination cannot change shape.
- For NCHW channel bias, use `[1,C,1,1]`.
- Trap: `[N,1] op [N]` produces `[N,N]`, not `[N,1]`.

13. Practice Tasks

1. Derive results for ten shape pairs, including incompatible and zero-sized cases.
2. Implement channel bias addition for NCHW and NHWC layouts.
3. Compare `expand` and `repeat` using storage size, strides, peak CUDA memory, and output equality.
4. Compute the gradient of a broadcast bias by hand and verify with autograd.
5. Find a pair of shapes that accidentally creates a quadratic outer operation.
6. Write a small CUDA-style pseudokernel that maps an output index to a zero-stride input.
7. Profile a broadcast add versus adding a materialized repeated tensor.
8. Test which in-place broadcast cases PyTorch accepts and explain each result.

14. Final Cheat Sheet

Item	Must remember
Core definition	Compatible singleton/missing dimensions are logically expanded.

Item	Must remember
Rule	Align right; sizes match or one is 1.
Representation	<code>expand</code> commonly uses stride 0.
Backward	Sum gradient over expanded dimensions.
Most asked	Output shape, <code>expand</code> vs <code>repeat</code> , in-place rules.
Interview trap	Broadcasting can silently create a huge unintended tensor.
One-line answer	"Broadcasting reuses singleton dimensions by stride, and backward reduces over them."

Autograd

1. Overview

Autograd is PyTorch's automatic differentiation engine. While tensor operations execute, autograd records the differentiable operations needed to compute derivatives. Calling `backward()` then applies the chain rule in reverse order, producing gradients for leaf tensors such as model parameters.

Autograd matters because modern training repeatedly computes a forward pass, a scalar loss, gradients, and parameter updates. In GPU systems, its graph also determines tensor lifetimes, backward kernel launches, synchronization behavior, and much of peak memory usage.

It is used in neural-network training, differentiable simulation, meta-learning, scientific optimization, and custom CUDA operators. Interviewers ask about it to see whether candidates understand dynamic computation graphs, reverse-mode differentiation, leaf tensors, saved activations, gradient accumulation, and in-place safety—not merely `.backward()` syntax.

2. Core Idea

Think of the forward pass as writing a receipt for every mathematical transformation. Each receipt says what operation occurred, which earlier results it depends on, and what information its derivative will need. Backward reads those receipts in reverse.

```
x = torch.tensor(3.0, device="cuda", requires_grad=True)
y = x * x
z = 2 * y + 1
z.backward()
print(x.grad)  # 12, because dz/dx = 2 * 2x
```

Step by step:

1. `x` is a leaf tensor requesting gradients.
2. `y=x*x` creates an autograd node whose backward needs `x`.
3. `z=2*y+1` creates later nodes.

4. `z.backward()` seeds $dz/dz=1$ because `z` is scalar.
5. Reverse-mode applies local vector-Jacobian products: $dz/dy=2$, then $dy/dx=2x$.
6. Contributions reaching leaf `x` accumulate into `x.grad`.

Autograd does not normally build a giant symbolic derivative expression. Each node implements a local **vector-Jacobian product (VJP)**. Reverse mode is efficient for a scalar loss with millions of parameters because it propagates one output cotangent backward.

3. Important Subtopics

Dynamic computation graph

PyTorch constructs the graph during each forward execution. Python control flow, loops, and conditional branches therefore record only the path actually taken.

Why it matters: graphs naturally support variable-length models and debugging, but Python/framework overhead can limit performance; compilation can capture and optimize graph regions.

Example: an `if x.sum() > 0` branch creates nodes for whichever branch executes.

Interview angle: "dynamic" means define-by-run, not that gradients are numerically approximated.

Leaf and non-leaf tensors

A user-created tensor with `requires_grad=True` and no differentiable creator is normally a leaf. Parameters are leaves; results of operations are non-leaves and have a `grad_fn`.

By default, `.grad` is populated for leaves. Use `.retain_grad()` if a non-leaf's gradient must be inspected.

Why it matters: candidates often expect every intermediate's `.grad` to remain populated.

Interview angle: distinguish gradient needed for propagation from gradient retained as a Python-visible field.

Backward graph and saved tensors

Backward formulas may save forward inputs or outputs. For $y=x*x$, backward needs `x`; other formulas may need `y` or metadata only.

Why it matters: saved activations dominate training memory. Gradient checkpointing trades extra forward computation for fewer saved activations.

Example: `torch.utils.checkpoint` discards selected intermediates and recomputes them in backward.

Interview angle: explain why training uses much more memory than inference.

Gradient accumulation

Leaf gradients accumulate with addition into `.grad`; `backward()` does not reset them automatically.

```
optimizer.zero_grad(set_to_none=True)
loss.backward()
```

```
optimizer.step()
```

Why it matters: accumulation enables large effective batches across microbatches, but forgetting to clear gradients changes the update.

Interview angle: `set_to_none=True` can avoid a fill-to-zero and lets code distinguish no gradient from a numerical zero gradient.

Grad mode, inference mode, and detach

`torch.no_grad()` disables recording for enclosed operations. `torch.inference_mode()` is stronger and can remove additional autograd tracking overhead for true inference. `.detach()` returns a tensor disconnected from the current graph while commonly sharing storage.

Why it matters: disabling graph construction reduces memory and CPU overhead, but it does not automatically synchronize or move tensors.

Interview angle: `model.eval()` changes module behavior such as dropout/batch normalization; it does **not** disable autograd.

In-place operations and version counters

Autograd tracks mutations with version counters. If backward needs a saved tensor that was modified in place, PyTorch can raise an error rather than silently using invalid data.

Why it matters: in-place operations do not always save memory because backward may need the original value, and aliases complicate mutation.

Interview angle: in-place is sometimes valid, not universally forbidden; correctness depends on derivative requirements and aliasing.

Higher-order derivatives and graph retention

The backward graph is normally freed as it is consumed. `retain_graph=True` preserves it for another traversal; `create_graph=True` records backward operations so higher-order derivatives can be computed.

Why it matters: both can sharply increase memory usage.

Interview angle: do not use `retain_graph=True` as a routine fix for graph-lifetime errors.

Custom autograd functions

`torch.autograd.Function` lets developers define paired static `forward` and `backward` methods. The context saves only data required by backward.

Why it matters: custom CUDA kernels need a mathematically correct derivative and correct device/stream behavior.

Interview angle: validate custom backward with `gradcheck` in double precision on small inputs when supported.

4. Real-World Example

A transformer training iteration illustrates the engine's role:

```
optimizer.zero_grad(set_to_none=True)

with torch.autocast("cuda", dtype=torch.float16):
    logits = model(tokens)
    loss = loss_fn(logits, targets)

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

During forward, matrix multiplications, normalization, activation, and attention nodes are recorded. Selected activations remain live. Backward launches GPU kernels in dependency order, may overlap independent branches, and accumulates parameter gradients. Distributed data parallel commonly launches gradient communication as buckets become ready, overlapping all-reduce with remaining backward compute.

If memory is insufficient, activation checkpointing can recompute transformer blocks during backward. The trade is deliberate: lower saved-tensor memory at the cost of more computation and kernel launches.

5. Diagrams / Mental Models

Forward graph:

x

----->

[multiply]

----->

y

----->

[scale]

----->

z(loss)

\

-----|

^

-----|

^

Backward traversal:

dz=1 --> scale backward --> dy --> multiply backward --> accumulate dx

Forward: parameters + inputs -> activations -> scalar loss

| saved |

Backward: parameter gradients <- VJPs <- seed gradient 1

Object	Role
<code>requires_grad</code>	Requests tracking for operations involving the tensor
<code>grad_fn</code>	Creator/backward node for a non-leaf result
<code>.grad</code>	Accumulated gradient retained mainly on leaves
saved tensor	Forward value retained for a backward formula

Object	Role
version counter	Detects invalidating in-place mutations

6. Common Interview Questions

1. **What is PyTorch autograd?** A reverse-mode automatic differentiation engine that records differentiable operations and evaluates VJPs backward. Expected: dynamic graph and chain rule. Mistake: calling it symbolic differentiation or finite differences.
2. **Why is reverse mode suitable for neural networks?** Training usually differentiates one scalar loss with respect to many parameters, which reverse mode handles in roughly a small constant multiple of forward work. Expected: contrast few outputs with many inputs. Mistake: saying it computes a full Jacobian explicitly.
3. **What is a leaf tensor?** A gradient-requiring tensor not produced by a tracked differentiable operation, such as a parameter. Expected: `.grad` accumulates on leaves by default. Mistake: calling the final loss a leaf.
4. **Why is an intermediate's `.grad` usually `None`?** Its gradient is used transiently for propagation but is not retained unless `.retain_grad()` is requested. Expected: distinguish propagation from retention. Mistake: concluding backward skipped the intermediate.
5. **Do gradients accumulate?** Yes, backward adds to existing leaf `.grad`. Expected: clear/set to `None` between independent steps or intentionally accumulate. Mistake: assuming replacement.
6. **What is saved during forward?** Only tensors/metadata each backward implementation needs, though collectively this can be large. Expected: saved tensors create memory lifetime. Mistake: claiming the entire Python execution is copied.
7. **What is the difference between `no_grad`, `inference_mode`, and `eval`?** The first two control autograd recording; inference mode applies stronger inference optimizations/restrictions; `eval()` changes module training behavior. Expected: separate graph recording from module state. Mistake: treating them as synonyms.
8. **What does `detach()` do?** It returns a tensor disconnected from the graph, usually sharing storage with the source. Expected: mutations can still alias. Mistake: saying it necessarily copies data.
9. **Why can an in-place operation break backward?** It can overwrite a saved value required for a derivative; version counters detect many cases. Expected: mention saved tensors, aliases, and version counters. Mistake: saying all in-place operations are banned or always save memory.
10. **What does `retain_graph=True` do?** It prevents the current graph from being freed after backward so it can be traversed again. Expected: memory cost. Mistake: using it to retain non-leaf `.grad`.
11. **How do you backpropagate from a non-scalar output?** Supply a `gradient` tensor of matching/broadcast-compatible semantics to compute a VJP, or reduce to a scalar. Expected: explain the upstream vector in a VJP. Mistake: expecting an implicit all-ones seed in every case.
12. **How does broadcasting affect backward?** Gradients are summed to the original input shape over broadcast axes. Expected: identify every expanded dimension. Mistake: returning the expanded gradient without reduction.
13. **Does CUDA backward execute synchronously?** Backward submits CUDA work to streams and is generally host-asynchronous like other CUDA operations; dependencies preserve correctness. Expected: separate host return from device completion. Mistake: assuming Python return always means all GPU kernels completed.
14. **How do you implement gradients for a custom CUDA op?** Provide a backward formula/custom op registration, save minimal required context, honor devices/layouts/streams, and test numerically.

Expected: cover mathematical and systems contracts. Mistake: differentiating only the common shape or ignoring non-contiguous inputs.

7. Deep-Dive Questions

- 1. What is a vector-Jacobian product?** Given output cotangent $\mathbf{v}$, a backward node computes $\mathbf{v}^T \mathbf{J}$ without materializing Jacobian $\mathbf{J}$. Composing VJPs implements reverse-mode chain rule efficiently.
- 2. How can autograd schedule independent graph branches?** A dependency count tracks when a node's output gradients are ready. Ready nodes enter engine queues; on CUDA their kernels are enqueued with stream dependencies. Exact scheduling is an implementation detail, so code should rely on dependencies, not assumed order between independent work.
- 3. Why does activation checkpointing preserve correct gradients?** It reruns a forward region during backward under controlled autograd state, recreating intermediates, then applies its derivatives. Random-state and side-effect behavior must be handled consistently.
- 4. What happens if a tensor contributes through two paths?** Each path produces a gradient contribution; autograd sums them at the shared ancestor, implementing the multivariable chain rule.
- 5. What must a custom backward return?** One gradient entry per forward input, using `None` for non-differentiable or unneeded inputs, with shapes/dtypes/devices consistent with the contract. Higher-order differentiation requires backward operations themselves to be differentiable or an explicit second-order strategy.

8. Comparison Tables

Reverse mode	Forward mode
Propagates output cotangents backward	Propagates input tangents forward
Efficient for few outputs, many inputs	Efficient for few inputs, many outputs
Core of ordinary <code>.backward()</code>	Useful for JVPs and some higher-order methods
Stores/interacts with reverse graph	Tracks tangent alongside primal computation

Mechanism	Tracks gradients?	Changes module behavior?	Typical use
Normal grad mode	Yes when required	No	Training
<code>torch.no_grad()</code>	No inside context	No	Evaluation snippets/updates
<code>torch.inference_mode()</code>	No, stronger restrictions/optimization	No	Pure inference
<code>model.eval()</code>	Unchanged	Yes	Dropout off, BatchNorm evaluation behavior
<code>detach()</code>	Disconnects returned tensor	No	Stop gradient across one value

9. Common Mistakes

- Forgetting to clear gradients between independent optimization steps.
- Believing `model.eval()` disables gradient tracking.
- Expecting `.grad` on every intermediate without `retain_grad()`.
- Using `retain_graph=True` indefinitely and causing a memory leak-like growth.
- Detaching a value accidentally with `.item()`, NumPy conversion, or explicit `detach()` in the middle of a needed path.
- Mutating saved tensors or their aliases in place.
- Assuming backward builds or stores the full Jacobian.
- Ignoring broadcast reductions in a custom backward.
- Saving more context than backward needs in a custom function.
- Timing CUDA backward without proper event synchronization.

10. Edge Cases / Special Cases

- Integer and Boolean tensors do not participate as differentiable floating-point/complex leaves in ordinary autograd.
- Non-scalar outputs require an explicit upstream gradient for `backward` unless reduced.
- Complex gradients follow PyTorch's complex differentiation conventions; conjugation details matter.
- A graph may be freed after backward even though output tensors still exist.
- `None` gradient and a tensor of zeros are semantically distinguishable to optimizers/hooks.
- Hooks can observe or transform gradients but may affect ordering and performance.
- Reentrant/custom backward, distributed collectives, and compilation introduce extra scheduling considerations.
- Anomaly detection is valuable for diagnosis but adds overhead and is not a production performance mode.
- Views share version counters/alias relationships relevant to in-place checks.

11. How to Explain in Interview

"PyTorch autograd builds a dynamic graph during the forward pass. Each operation contributes a backward node that computes a local vector-Jacobian product and may save the minimum forward data it needs. Calling backward seeds the loss gradient and traverses dependencies in reverse, summing contributions into leaf gradients. Saved activations drive training memory, gradients accumulate by default, and version counters protect backward from invalid in-place mutation."

12. Quick Revision Notes

- Dynamic define-by-run graph.
- Reverse mode composes VJPs; no full Jacobian is normally built.
- Parameters are leaves; results have `grad_fn`.
- `.grad` accumulates on leaves.
- Saved tensors create training memory pressure.
- `eval()` is not `no_grad()`.
- `detach()` stops graph connectivity but often shares storage.
- `retain_graph` keeps a graph; `create_graph` enables higher derivatives.
- Broadcast backward sums to original shape.
- Trap: CUDA backward submission is generally asynchronous with respect to the CPU.

13. Practice Tasks

1. Draw the graph and calculate gradients for `z=(x*y + x).sum()` by hand.
2. Call backward twice with and without clearing `.grad` and explain the values.
3. Inspect `is_leaf`, `grad_fn`, and `.grad` for parameters and intermediates.
4. Compare memory in training, `no_grad`, and `inference_mode`.
5. Trigger and explain a version-counter error by mutating a saved tensor.
6. Write a custom `autograd.Function` for a simple fused multiply-add and run `gradcheck`.
7. Implement microbatch gradient accumulation and verify equivalence after correct loss scaling.
8. Profile a backward pass and match backward kernels to forward operators.
9. Add activation checkpointing to a block and measure memory-versus-time tradeoff.

14. Final Cheat Sheet

Item	Must remember
Core definition	Dynamic reverse-mode automatic differentiation using VJPs.
Main memory cost	Forward tensors saved for backward.
Gradient rule	Leaf <code>.grad</code> accumulates contributions.
Most asked	Leaves, saved tensors, detach/no-grad/eval, in-place safety.
Common comparison	Reverse mode: few outputs/many inputs; forward mode: few inputs/many outputs.
Interview trap	<code>eval()</code> alone still records autograd.
One-line answer	"Autograd records local derivative nodes in forward and composes their VJPs backward."

CUDA Tensors

1. Overview

A **CUDA tensor** is a PyTorch tensor whose storage resides in memory managed for an NVIDIA CUDA device and whose supported operations dispatch to CUDA implementations. It retains the same tensor abstraction—shape, strides, dtype, and autograd metadata—but computation and memory access obey CUDA's asynchronous, device-specific execution model.

CUDA tensors matter because simply moving data to a GPU does not guarantee speed. Transfers, dtype, layout, batch size, synchronization, kernel support, and device placement all determine performance and correctness. They are used in model training/inference, scientific computing, image pipelines, and GPU-accelerated data processing.

Interviewers ask about CUDA tensors to test host-versus-device memory, transfer semantics, device mismatch, asynchronous execution, pinned memory, multi-GPU behavior, and correct performance measurement.

2. Core Idea

Think of CPU and GPU as two workshops with separate benches. A tensor's `device` says which bench holds its data. A CUDA kernel can efficiently work on the GPU bench; Python and ordinary CPU code cannot directly treat that device allocation like a normal CPU array.

```
cpu_x = torch.randn(1024, 1024)
gpu_x = cpu_x.to("cuda")
gpu_y = gpu_x @ gpu_x
cpu_y = gpu_y.to("cpu")
```

Step by step:

1. `cpu_x` is allocated in host memory.
2. `.to("cuda")` allocates device storage and transfers data.
3. Matrix multiplication dispatches a CUDA/backend kernel and returns after asynchronous submission in the ordinary case.
4. `.to("cpu")` must eventually make results available in host memory and commonly introduces a host-visible wait for an ordinary blocking copy.
5. CPU and CUDA tensors cannot generally be mixed in one arithmetic operation because a kernel needs operands accessible on its device.

The performance goal is usually to move sizable data to the GPU, perform enough GPU work to amortize transfer and launch overhead, and avoid unnecessary round trips.

3. Important Subtopics

Device placement and dispatch

`tensor.device` identifies CPU or a CUDA device such as `cuda:0`. An operator dispatches based on tensor properties to an appropriate kernel implementation.

Why it matters: model parameters, inputs, temporary states, and newly created tensors must be on compatible devices.

Example: prefer `torch.zeros_like(x)` or `x.new_zeros(...)` when the new tensor should inherit `x`'s device and dtype.

Interview angle: device mismatch is a correctness error, not an implicit transfer opportunity in most tensor arithmetic.

Host-to-device and device-to-host copies

`.to`, `.cuda`, `.cpu`, and `.copy_` can transfer data. Transfers are expensive relative to many small operations and may synchronize.

Why it matters: a GPU pipeline that copies each scalar or small array separately can be slower than CPU execution.

Example: batch examples on the CPU, then transfer one larger tensor.

Interview angle: PCIe/NVLink transfer is different from GPU global-memory bandwidth.

Pinned host memory and `non_blocking`

Page-locked (pinned) host memory enables DMA transfers without first staging pageable pages and is important for true asynchronous host-device copies. A `DataLoader` can use `pin_memory=True`; `.to(device, non_blocking=True)` can then enqueue eligible transfers.

Why it matters: pinned buffers can overlap transfer with compute when streams and dependencies are arranged correctly.

Interview angle: `non_blocking=True` is an opportunity, not a blanket concurrency guarantee; excessive pinning harms host memory management.

Dtypes and mixed precision

CUDA devices have different throughput/support for `float32`, `float16`, `bfloat16`, TF32-related matmul modes, integers, and `float64`. Autocast selects suitable lower-precision operations; gradient scaling protects small `float16` gradients from underflow.

Why it matters: dtype affects storage, bandwidth, Tensor Core eligibility, numerical range, and output accuracy.

Interview angle: mixed precision is not “convert everything to FP16.” Accumulation and sensitive operations may use higher precision.

Asynchrony and synchronization

CUDA operations are commonly enqueued and return before device completion. Operations in the same stream are ordered; dependent PyTorch operations are correct without a device-wide wait.

Host-visible operations such as `.item()`, some device-to-host copies, printing tensor values, or explicit `torch.cuda.synchronize()` can force waiting.

Why it matters: hidden synchronization harms overlap and makes naive timing wrong.

Interview angle: distinguish submission time from GPU execution time.

Multi-GPU placement and peer communication

`cuda:0` and `cuda:1` are distinct devices with separate allocations and stream contexts. Moving between them can use peer access/interconnect paths when supported, but is not free.

Why it matters: distributed training partitions data/model states and communicates gradients/activations.

Interview angle: `CUDA_VISIBLE_DEVICES` can remap visible device indices; code should not assume physical numbering.

Lifetime and allocator behavior

Deleting the last Python reference makes a CUDA allocation eligible for PyTorch's caching allocator; it does not necessarily return memory to the driver immediately. CUDA work using a tensor must also be accounted for across streams.

Why it matters: `nvidia-smi` can show reserved memory after tensors are deleted.

Interview angle: allocated-by-live-tensors, reserved-by-allocator, and total device use are different metrics.

4. Real-World Example

A high-throughput input pipeline overlaps CPU decoding, pinned-memory transfer, and model execution:

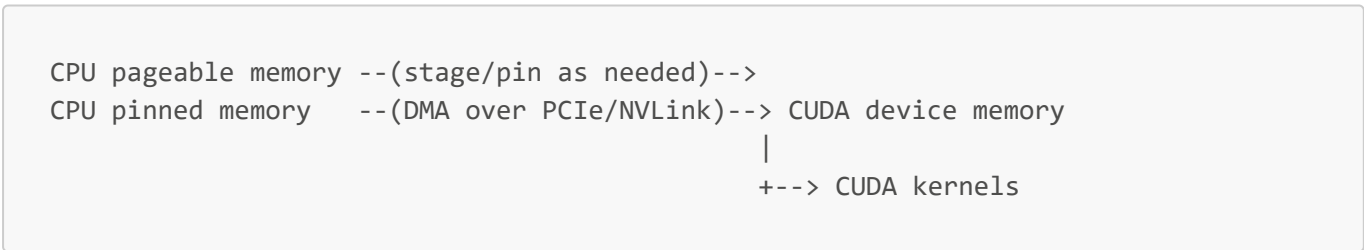
```
loader = torch.utils.data.DataLoader(
    dataset, batch_size=128, num_workers=8, pin_memory=True
)

model = model.to("cuda").eval()
with torch.inference_mode():
    for images, labels in loader:
        images = images.to("cuda", non_blocking=True)
        logits = model(images)
```

For meaningful overlap, workers must prepare batches early, pinned memory must be available, and transfer/compute stream organization must avoid unnecessary dependencies. The batch must be large enough that GPU efficiency outweighs launch and transfer costs.

Calling `logits.cpu()` every iteration solely for intermediate inspection would add transfers and synchronization. A production server instead keeps preprocessing, model layers, and postprocessing on the GPU when beneficial, transferring only final compact results.

5. Diagrams / Mental Models



Bad fine-grained flow:
CPU -> GPU small copy -> tiny kernel -> CPU copy -> repeat

Better batched flow:
CPU batch -> GPU copy -> many GPU kernels / fused graph -> compact result -> CPU

Observation	What it means
<code>tensor.device == cuda:0</code>	Storage is associated with visible CUDA device 0
<code>memory_allocated()</code>	Bytes held by live tensors (roughly allocator accounting)
<code>memory_reserved()</code>	Bytes reserved in allocator-managed segments

Observation	What it means
<code>nvidia-smi</code> use	Broader context/driver-visible process memory; not equal to live tensors

6. Common Interview Questions

1. **What is a CUDA tensor?** A tensor with storage on a CUDA device whose operations dispatch to CUDA kernels when supported. Expected: same strided abstraction, different device/execution model. Mistake: calling it a CPU tensor merely “processed faster.”
2. **How do you move a tensor to the GPU?** Use `.to(device)` or `.cuda()`, ideally in batches and without redundant copies. Expected: assignment may be needed because these return tensors. Mistake: assuming `.to()` always mutates the original.
3. **Can CPU and CUDA tensors be added directly?** Generally no; operands must be on compatible devices. Expected: explicitly move data. Mistake: expecting silent automatic transfer.
4. **Why might a GPU version be slower?** Small workload, transfer cost, launch overhead, synchronization, unsuitable layout/dtype, or inefficient kernel. Expected: profile the complete pipeline. Mistake: blaming GPU clock alone.
5. **What is pinned memory?** Page-locked host memory suitable for efficient DMA and eligible asynchronous transfers. Expected: it is a limited host resource. Mistake: calling it GPU memory.
6. **What does `non_blocking=True` guarantee?** It requests an asynchronous-capable copy where conditions permit; it does not guarantee useful overlap. Expected: pinned host memory and stream scheduling matter. Mistake: assuming the copy has completed when the call returns.
7. **Why is `.item()` expensive in a GPU loop?** The CPU needs a concrete scalar, often forcing synchronization and a device-to-host transfer. Expected: aggregate on device or reduce frequency. Mistake: treating it as a free Python conversion.
8. **How should CUDA operations be timed?** Use CUDA events on the relevant stream or synchronize around wall-clock measurement; include warm-up and define whether transfers are included. Expected: state the measured scope and synchronization boundary. Mistake: timing only asynchronous launch calls.
9. **What does mixed precision improve?** It can reduce storage/bandwidth and use high-throughput Tensor Core paths while retaining appropriate higher-precision work. Expected: numerical tradeoffs and gradient scaling. Mistake: converting every operation and accumulator blindly.
10. **Why does `nvidia-smi` still show memory after `del tensor`?** PyTorch's caching allocator retains freed blocks for reuse, and context/library allocations also exist. Expected: reserved versus allocated. Mistake: immediately labeling it a leak.
11. **How do you create a tensor on the same device as another?** Use `*_like`, `new_*`, or pass `device=x.device` and usually `dtype=x.dtype`. Expected: inherit both device and relevant dtype/layout. Mistake: creating CPU constants in a CUDA hot path.
12. **What changes in multi-GPU code?** Device placement, peer/collective communication, per-device streams, and synchronization must be explicit. Expected: discuss placement, topology, and communication. Mistake: treating all GPUs as one shared memory pool.
13. **Does a CUDA kernel finish when the Python operation returns?** Usually not; submission is asynchronous with respect to the host. Expected: stream ordering preserves dependent device work. Mistake: reading host wall time as kernel duration.

7. Deep-Dive Questions

1. **When is a host-to-device copy truly overlap-capable?** The source is normally pinned, the copy is enqueued asynchronously, there is no dependency forcing serialization, the device has suitable copy/compute engines, and another stream has independent compute to overlap.
2. **What happens when you call `.numpy()` on a CUDA tensor?** NumPy expects host-accessible memory, so direct conversion is not allowed; move to CPU first, which may synchronize and copy. If gradients are tracked, detach according to intended semantics.
3. **Why are same-stream dependent operations correct without `synchronize()`?** CUDA stream order ensures later operations observe earlier writes in that stream, and PyTorch inserts/uses required dependencies for its managed operations. Synchronization is needed only at boundaries requiring host or cross-stream visibility beyond established dependencies.
4. **How can dtype influence kernel choice?** Dispatch selects dtype-specific implementations; matrix dimensions/alignment and device capability may enable Tensor Cores or require fallback paths. Numerical policies can also change accumulation precision.
5. **Why is copying a model to CUDA inside every request harmful?** It repeatedly allocates and transfers parameters, prevents stable reuse/caching, and adds synchronization. Keep long-lived parameters and reusable buffers on the target device.

8. Comparison Tables

CPU tensor		CUDA tensor	
Host memory		Device-associated memory	
CPU kernels		CUDA/library kernels	
Ordinary host code can access values directly		Host value access normally requires transfer/synchronization	
Operations often appear host-synchronous		Most GPU work is host-asynchronous	
Better for tiny/branch-heavy workloads in many cases		Better for sufficiently parallel/batched workloads	
Pageable host memory	Pinned host memory	Device memory	
Default host allocation	Page-locked host allocation	GPU-accessible allocation	
OS can page it	Cannot be paged while pinned	Managed by CUDA/device allocator	
Async DMA often needs staging	Supports efficient async transfer	High-bandwidth kernel workspace	
Cheap/general allocation	Scarce; overuse hurts system	Limited by GPU capacity	

9. Common Mistakes

- Moving individual small tensors back and forth inside a hot loop.
- Forgetting that `.to(device)` returns a tensor rather than always mutating.
- Creating CPU constants during CUDA computation.
- Timing asynchronous calls without events/synchronization.
- Calling `.item()` or printing GPU tensors every iteration.
- Assuming `non_blocking=True` alone guarantees overlap.
- Overusing pinned host memory.

- Converting everything to FP16 without numerical validation.
- Treating reserved allocator memory as a leak.
- Assuming visible device 0 is always physical GPU 0.
- Forgetting warm-up effects from context creation, library initialization, and compilation/autotuning.

10. Edge Cases / Special Cases

- Some operations lack implementations for certain CUDA dtypes/layouts and may error rather than fall back silently.
- Cross-device tensor operations have specific support; ordinary arithmetic generally requires one device.
- Unified/managed-memory behavior may appear in extensions but ordinary PyTorch CUDA tensor practice still requires device-aware reasoning.
- CUDA context initialization makes the first operation unusually expensive.
- Asynchronous errors may surface at a later synchronization/API call, obscuring the original source.
- Non-default streams require explicit lifetime/dependency handling for tensors.
- Forked multiprocessing after CUDA initialization is unsafe in common setups; use supported process-start patterns.
- Deterministic algorithm settings can select slower kernels or reject nondeterministic operations.
- A tensor's logical element bytes are not the entirety of process GPU memory; allocator metadata, workspaces, contexts, and libraries contribute.

11. How to Explain in Interview

"A CUDA tensor has strided storage on a specific CUDA device, and PyTorch dispatches supported operations to GPU kernels. GPU work is usually asynchronous to the CPU, so I keep tensors on-device, batch enough work to amortize launches and transfers, use pinned memory plus nonblocking copies when building overlap, and time with CUDA events. I also separate live allocated memory from allocator-reserved and context memory."

12. Quick Revision Notes

- Device is part of a tensor's execution contract.
- `.to()` may copy and returns the resulting tensor.
- Keep operands on compatible devices.
- Batch transfers and avoid GPU↔CPU ping-pong.
- Pinned host memory enables efficient async DMA but is limited.
- CUDA operations are commonly host-asynchronous.
- `.item()` often synchronizes.
- Use events or explicit synchronization for correct timing.
- Mixed precision is selective and numerically aware.
- Trap: `nvidia-smi` memory is not equal to live tensor bytes.

13. Practice Tasks

1. Benchmark CPU versus GPU addition across sizes and locate the crossover including transfers.
2. Compare pageable and pinned DataLoader transfers with `non_blocking=True`.
3. Demonstrate incorrect wall-clock timing and correct it with CUDA events.
4. Profile the effect of `.item()` inside versus outside an iteration loop.

5. Compare FP32, autocast FP16, and BF16 for speed, memory, and numerical error.
6. Print allocated, reserved, and driver-visible memory before and after allocations/deletions.
7. Write device-agnostic code using `*_like` and explicit device arguments.
8. Run two-device transfer experiments if hardware is available and describe topology effects.

14. Final Cheat Sheet

Item	Must remember
Core definition	Strided tensor storage associated with a CUDA device.
Main cost boundary	Host-device transfer and synchronization.
Performance rule	Keep data on GPU and submit sufficiently large work.
Most asked	Pinned memory, async timing, device mismatch, mixed precision.
Common comparison	CPU direct access versus CUDA asynchronous device execution.
Interview trap	Python return means submitted, not necessarily GPU-complete.
One-line answer	"A CUDA tensor moves the tensor abstraction to device storage, so placement, transfer, and stream semantics become part of correctness and speed."

CUDA Streams in PyTorch

1. Overview

A **CUDA stream** is an ordered queue of GPU operations. Operations submitted to one stream execute in issue order; operations in different streams may overlap when dependencies and hardware resources allow. PyTorch uses a current stream per device and exposes stream/event APIs for advanced scheduling.

Streams matter for overlapping copies with compute, pipelining independent model work, and coordinating custom CUDA extensions. They are used in input pipelines, multi-stage inference, distributed communication, and framework internals.

Interviewers ask about streams because candidates often confuse asynchronous submission with concurrency, assume the default stream is globally sufficient, or forget cross-stream tensor lifetime and synchronization.

2. Core Idea

Think of streams as conveyor belts feeding one factory. Items on one belt remain ordered. Two belts give the factory permission to work on independent items concurrently, but concurrency occurs only if machines and resources are available.

```
s = torch.cuda.Stream()
x = torch.randn(4096, 4096, device="cuda")
```

```
with torch.cuda.stream(s):
    y = x.square()
    z = y.sum()

# Work on s was enqueued; the current/default stream must wait before consuming z.
torch.cuda.current_stream().wait_stream(s)
result = z * 2
```

Step by step:

1. Create a stream on a CUDA device.
2. Entering the context makes it current for PyTorch CUDA operations in that block.
3. `square` and `sum` are enqueued in order on `s`.
4. Leaving the context restores the prior current stream; it does not wait for `s`.
5. `wait_stream(s)` inserts a device-side dependency into the current stream.
6. Later current-stream work can safely consume `z` without blocking the CPU globally.

3. Important Subtopics

Current and default streams

Each CUDA device has stream state; PyTorch operations normally enqueue on that device's current stream. The default stream is the initial current stream, but code can temporarily select another.

Why it matters: "the stream" is not one process-wide Python queue, especially in multi-device/multi-thread scenarios.

Example: `torch.cuda.current_stream(device)` inspects the current stream.

Interview angle: custom extensions should launch on the current stream rather than hard-coding the legacy default stream.

Ordering versus concurrency

Within a stream, issued operations are ordered. Across streams, no automatic total order exists; overlap is possible but not guaranteed.

Why it matters: dependencies, SM occupancy, memory bandwidth, copy engines, and kernel duration determine actual concurrency.

Interview angle: two streams are a scheduling mechanism, not two GPUs.

Events and stream waits

A CUDA event represents a point in a stream. Another stream can wait on it, or the host can query/synchronize it. Events are also used for GPU timing.

```
event = torch.cuda.Event()
with torch.cuda.stream(producer):
    value = make_value()
```

```
event.record()
consumer.wait_event(event)
with torch.cuda.stream(consumer):
    use(value)
```

Why it matters: an event creates a narrow device-side dependency instead of blocking the whole host/device.

Interview angle: prefer precise waits over `torch.cuda.synchronize()` when possible.

Copy/compute overlap

Pinned host memory, asynchronous copy APIs, a non-default stream, and independent compute can enable overlap. Hardware copy-engine support and direction also matter.

Why it matters: input transfer for batch $N+1$ can overlap compute for batch N .

Interview angle: explain all prerequisites, not merely “use two streams.”

Tensor lifetime and `record_stream`

PyTorch's caching allocator normally reasons about the stream on which a tensor was created/used. If a tensor is used on another stream, `tensor.record_stream(stream)` can tell the allocator not to reuse its memory until that stream's work completes.

Why it matters: Python lifetime can end before asynchronous GPU use finishes, and the allocator could otherwise recycle memory too early.

Interview angle: synchronization protects data dependencies; `record_stream` protects allocator lifetime. Often both concepts must be considered.

Autograd stream semantics

Backward operations follow stream relationships associated with their forward work and insert needed dependencies. When manually mixing streams, the caller must still synchronize before consuming gradients/results on another stream.

Why it matters: old habits of relying on broad default-stream synchronization can be incorrect or over-synchronizing.

Interview angle: use documented stream dependencies, not assumptions about global default-stream barriers.

Stream priorities and graphs

CUDA streams can have priorities that influence scheduling preference, not hard real-time guarantees. CUDA Graph capture records a fixed sequence of work for low-overhead replay and imposes stricter allocation/control-flow rules.

Why it matters: priorities help latency-sensitive work; graphs reduce repeated CPU launch overhead.

Interview angle: priority does not preempt every running kernel, and a CUDA graph is not the same as an autograd graph.

4. Real-World Example

A double-buffered inference pipeline prepares two pinned host buffers and uses a transfer stream plus the compute stream:

```
Time --->
transfer stream: H2D batch 0 | H2D batch 1 | H2D batch 2 |
compute stream:               infer batch 0 | infer batch 1 |
```

For each slot:

- 1. Wait until its previous GPU use is finished before refilling host/device buffers.
- 2. Enqueue H2D on the transfer stream.
- 3. Record an event after the copy.
- 4. Make the compute stream wait for that event.
- 5. Enqueue inference.
- 6. Record lifetime on every stream that uses allocator-backed tensors.

If inference saturates memory bandwidth or all SM resources, copy overlap may be limited. A profiler timeline—not stream count—reveals whether the pipeline overlaps.

5. Diagrams / Mental Models

```
Producer stream: [write X] --record E----->
                  |
Consumer stream: -----wait E--> [read X] ---->

The host enqueues both; the event creates device-side ordering.
```

```
Same stream:      A -> B -> C          guaranteed issue-order dependency
Different streams: A ----->
                  B ----->          may overlap; add waits for dependencies
```

Tool	Scope	Host blocks?	Typical purpose
<code>stream.wait_event(e)</code>	Future work in one stream	No	Cross-stream dependency
<code>stream.wait_stream(s)</code>	Future work waits for prior work in <code>s</code>	No	Join producer stream
<code>event.synchronize()</code>	Host waits for one event	Yes	Read result/timing boundary
<code>stream.synchronize()</code>	Host waits for one stream	Yes	Debug/boundary

Tool	Scope	Host blocks?	Typical purpose
<code>torch.cuda.synchronize()</code>	Host waits for device work	Yes	Broad debug/timing boundary

6. Common Interview Questions

1. **What is a CUDA stream?** An ordered queue of device operations. Expected: same-stream order, possible cross-stream overlap. Mistake: calling it a CPU thread or independent GPU.
2. **Do two streams guarantee concurrent kernels?** No. Hardware resources, dependencies, and kernel characteristics decide overlap. Expected: distinguish permission to overlap from observed overlap. Mistake: equating separate queues with simultaneous execution.
3. **What stream does PyTorch use?** Operations enqueue on the current stream for the relevant device; initially that is a default stream. Expected: stream contexts can change it. Mistake: saying PyTorch always uses one immutable stream.
4. **Does leaving with `torch.cuda.stream(s)` synchronize?** No; it restores the previous current stream. Expected: add a dependency before cross-stream consumption. Mistake: treating context exit as a join.
5. **How do you synchronize two streams efficiently?** Record an event in the producer and make the consumer wait, or use `wait_stream` for prior producer work. Expected: device-side wait. Mistake: device-wide synchronization.
6. **What is a CUDA event?** A marker recorded at a stream position, used for dependencies, completion queries, or timing. Expected: explain that it completes after prior stream work. Mistake: treating event recording as an immediate host wait.
7. **What is `record_stream` for?** It informs the caching allocator that a tensor's storage is in use by another stream, delaying safe reuse. Expected: allocator lifetime. Mistake: describing it as a compute dependency by itself.
8. **What is required to overlap H2D copy and compute?** Pinned host memory, async copy, suitable separate streams, independence, and hardware engine/resource capability. Expected: list both software and hardware prerequisites. Mistake: using pageable memory and merely setting `non_blocking=True`.
9. **Why can streams make code incorrect?** A consumer can read before a producer finishes, or storage can be reused while another stream still accesses it. Expected: data dependency and lifetime dependency. Mistake: considering only kernel order.
10. **Why can too many streams hurt?** More scheduling/launch overhead, resource contention, memory lifetime, and complexity; concurrent kernels may interfere. Expected: mention contention and longer lifetimes. Mistake: assuming more queues mean linear speedup.
11. **How do you time work on a stream?** Record start/end CUDA events in that stream, synchronize the end event, and read elapsed time. Expected: place both events around the target work on its stream. Mistake: record events on an unrelated stream.
12. **What is stream priority?** A scheduling hint/preference for pending work. Expected: not a deadline or universal preemption guarantee. Mistake: promising immediate interruption of running kernels.
13. **How does autograd interact with custom streams?** Autograd tracks/inserts dependencies for recorded operations, but callers must synchronize streams when accessing results/gradients elsewhere. Expected: mention forward/backward stream relationships and caller-side consumption. Mistake: assuming backward globally synchronizes all work.

7. Deep-Dive Questions

1. **Why isn't same-stream synchronization needed between dependent PyTorch ops?** FIFO stream semantics ensure the consumer begins after earlier producer commands in that stream. Avoiding host waits is central to asynchronous pipelines.
2. **What is the difference between a data race and premature allocator reuse?** A data race comes from missing execution order between accesses. Premature reuse occurs when the allocator believes a block is free while queued work on another stream still uses it. Event waits and `record_stream` address related but distinct responsibilities.
3. **Can a short kernel overlap a long kernel?** Possibly, if the long kernel leaves SM resources and scheduling opportunity. A fully occupying kernel may prevent meaningful concurrent execution despite different streams.
4. **Why might copy and compute fail to overlap?** Pageable source memory, same-stream ordering, implicit synchronization, dependency on the copied data, saturated interconnect/memory system, or unavailable copy engines can serialize them.
5. **How do CUDA Graphs relate to streams?** Capture observes work submitted to a capture stream and allowed participating streams, records dependencies, then replay submits the captured graph with much lower CPU overhead. Addresses and control flow often need to remain stable across replays.

8. Comparison Tables

Same stream	Different streams
Operations ordered by issue	No total order unless dependency added
Easy correctness for producer/consumer	Enables possible overlap
Less scheduling complexity	Requires data/lifetime reasoning
Cannot overlap ordered operations	Can overlap if hardware permits
Stream	CPU thread
CUDA device command queue	Host execution context scheduled by OS
Orders GPU operations	Executes host instructions
Many streams may feed one GPU	Threads can submit to streams
Concurrency depends on device resources	Concurrency depends on CPU cores/scheduler

9. Common Mistakes

- Assuming a stream is a GPU hardware core.
- Assuming two streams guarantee overlap.
- Forgetting that leaving a stream context does not wait.
- Using device-wide synchronization for every dependency.
- Consuming another stream's result without an event/wait.
- Forgetting `record_stream` when allocator-backed storage crosses streams.
- Confusing `record_stream` with execution ordering.
- Measuring one stream with events recorded on another.
- Attempting copy/compute overlap from pageable host memory.
- Creating many streams without profiling resource contention.

- Hard-coding the default stream in a custom CUDA extension.

10. Edge Cases / Special Cases

- Default-stream semantics can differ by legacy/per-thread modes at lower CUDA layers; PyTorch code should rely on explicit current-stream and dependency APIs.
- An event recorded before work is not equivalent to an event recorded after it.
- A wait affects future commands enqueued in the waiting stream, not already submitted earlier commands.
- Stream priorities are hints and do not provide strict fairness or deadlines.
- Asynchronous failures can be reported at later waits or unrelated-looking API boundaries.
- Cross-device events/waits and peer operations have device-specific constraints.
- Capturing CUDA Graphs restricts synchronization, allocation, data-dependent control flow, and address changes.
- Library handles and custom extensions must be correctly associated with the current stream.
- Allocator event tracking can delay block reuse and temporarily increase reserved memory.

11. How to Explain in Interview

"A CUDA stream is an ordered device work queue. Same-stream dependencies need no host synchronization; different streams can overlap but require explicit events or waits for producer-consumer order. In PyTorch I launch custom work on the current stream, use pinned memory and a transfer stream for eligible overlap, and call `record_stream` when allocator-backed tensors are used across streams so memory is not recycled too early."

12. Quick Revision Notes

- One stream: ordered operations.
- Multiple streams: possible, not guaranteed, overlap.
- Stream context exit does not synchronize.
- Events express narrow device-side dependencies and timing.
- Prefer stream/event waits to device-wide waits.
- Copy overlap needs pinned memory and hardware support.
- `record_stream` handles allocator lifetime, not data ordering alone.
- Custom kernels should use the current PyTorch stream.
- Trap: asynchronous submission is not concurrent execution.

13. Practice Tasks

1. Launch independent matrix operations on two streams and inspect a profiler timeline.
2. Intentionally omit a stream wait, then fix the producer-consumer dependency with an event.
3. Build a pinned-memory transfer/compute double buffer.
4. Compare `wait_event`, `wait_stream`, stream synchronization, and device synchronization.
5. Measure a kernel with events on the correct and an incorrect stream.
6. Demonstrate safe cross-stream temporary use with `record_stream`.
7. Test whether two memory-bound kernels overlap usefully versus two small compute kernels.
8. Capture a static workload with `torch.cuda.CUDAGraph` and compare CPU launch overhead.

14. Final Cheat Sheet

Item	Must remember
Core definition	Ordered queue of CUDA operations.
Main guarantee	Issue order within one stream.
Cross-stream rule	Add event/wait for dependencies; overlap is only possible.
Lifetime rule	Record cross-stream tensor use for allocator safety.
Most asked	Events, synchronization scope, copy overlap, current stream.
Interview trap	A stream context restores state; it does not join work.
One-line answer	"Streams provide order within a queue and optional overlap across queues, with explicit dependencies and lifetime tracking."

Memory Allocator

1. Overview

PyTorch uses a **CUDA caching allocator** to manage most CUDA tensor storage. Instead of asking the CUDA driver for a fresh allocation and returning it on every tensor creation/destruction, it reserves larger memory segments, splits them into blocks, and reuses freed blocks.

This matters because low-level device allocation/free can be expensive and can introduce synchronization. Caching makes repeated training iterations fast, but it also means unused cached memory may remain visible as process memory in tools such as `nvidia-smi`. Fragmentation, tensor lifetime, autograd saves, workspaces, and cross-stream use all affect whether a request can be satisfied.

The allocator is used behind almost every ordinary CUDA tensor, intermediate, gradient, and optimizer state. Interviewers ask about it to distinguish a real leak from caching, explain out-of-memory errors, and test understanding of allocated versus reserved memory, fragmentation, stream safety, and `empty_cache()`.

2. Core Idea

Think of the allocator as a warehouse. The CUDA driver gives PyTorch warehouse floors (segments). PyTorch divides them into shelves (blocks) for tensors. When a tensor dies, its shelf becomes reusable; the warehouse usually keeps the floor instead of giving it back immediately.

```
import torch, gc

x = torch.empty(256 * 1024 * 1024, dtype=torch.uint8, device="cuda")
print(torch.cuda.memory_allocated())
print(torch.cuda.memory_reserved())
```

```
del x
gc.collect()
print(torch.cuda.memory_allocated()) # decreases when no live tensor owns it
print(torch.cuda.memory_reserved()) # may remain high for reuse
```

Step by step:

1. A tensor requests a block of bytes.
2. The allocator searches reusable free blocks in its pools.
3. If a larger free block is chosen, it may split the block.
4. If no suitable block exists, it obtains/reserves more memory from CUDA.
5. When the tensor becomes unreachable and queued use is safe, the block becomes free in the cache.
6. A later compatible request can reuse it quickly.
7. `empty_cache()` may release currently unused cached blocks/segments to CUDA, but cannot free live tensors.

3. Important Subtopics

Allocated versus reserved memory

Allocated memory is memory occupied by live tensor allocations as tracked by the allocator. **Reserved** memory is the larger pool held by the allocator, including allocated blocks and cached free blocks.

Why it matters: `reserved >= allocated` in the ordinary accounting model, and the gap is not automatically a leak.

Example: after deleting a 1 GiB tensor, allocated can fall while reserved remains available to the next iteration.

Interview angle: also mention memory outside this allocator—CUDA context, library workspaces, NCCL, custom extensions.

Blocks, segments, splitting, and coalescing

Allocator implementations acquire segments and service requests with blocks. Larger free blocks can be split; adjacent compatible free blocks may be merged.

Why it matters: repeated irregular sizes can leave free space in pieces that cannot satisfy one large contiguous request.

Example: 2 GiB total free across many small blocks may not satisfy one 1 GiB request if no suitable segment/block can be formed and device capacity prevents growth.

Interview angle: fragmentation is about usable block shapes, not just the sum of free bytes.

Small and large allocation behavior

The allocator uses size classes/pools and rounding policies to balance reuse against wasted space. Exact thresholds and configuration are implementation/version details.

Why it matters: stable shapes lead to stable reuse; highly variable batch/sequence sizes can create many allocation sizes.

Example: bucketing sequences into a few padded lengths often improves allocator reuse as well as kernel regularity.

Interview angle: explain the principle without depending on undocumented threshold numbers.

Stream-aware reuse and deferred freeing

CUDA work is asynchronous. A block cannot be reused while any queued kernel still accesses it. The allocator tracks stream use, often with events; `record_stream` helps when a tensor crosses streams.

Why it matters: deleting a Python object does not prove the GPU is finished with its storage.

Example: a temporary produced on stream A and consumed on B must have correct dependency/lifetime tracking before its block can be recycled.

Interview angle: Python reference lifetime and device-use lifetime are different.

Out-of-memory diagnosis

OOM can result from genuinely live tensors, fragmentation, transient workspace/peak overlap, retained graphs, growing caches in user data structures, or another process.

Why it matters: the fix depends on the cause. Smaller batches help capacity pressure but not a Python list that retains graphs forever.

Useful tools include `memory_allocated`, `max_memory_allocated`, `memory_reserved`, `memory_stats`, `memory_summary`, snapshots/history APIs available in the installed PyTorch version, and the profiler.

Interview angle: gather a timeline and reference/lifetime evidence before proposing `empty_cache()`.

`empty_cache()`

`torch.cuda.empty_cache()` releases eligible unused cached memory so it can be used by other CUDA consumers. It does not release live tensor memory and usually does not increase the amount PyTorch could use for live tensors in a simple steady state.

Why it matters: calling it every iteration can force costly driver allocations later and reduce performance.

Interview angle: useful at phase boundaries or when sharing the GPU, not a general leak fix.

Autograd, optimizer, and workspace memory

Training memory includes parameters, gradients, optimizer states, saved activations, temporary outputs, and library/compiler workspaces. Peak memory depends on overlapping lifetimes, not simply their final totals.

Why it matters: Adam-like optimizers maintain extra state; backward may hold activations while gradients appear; fused/compiled kernels can change temporary needs.

Interview angle: provide a category breakdown rather than only "model size times N."

Allocator configuration and alternate backends

PyTorch exposes allocator environment configuration and, in supported setups, may use different allocation strategies/backends. Exact option names and behavior should be checked for the installed version.

Why it matters: configuration can mitigate a known workload pattern but is secondary to fixing unnecessary lifetimes and shape churn.

Interview angle: do not present tuning knobs as universal defaults.

4. Real-World Example

A variable-length language-model server receives batches with sequence lengths from 8 to 8192. Each shape requires different attention buffers. Without bucketing, the allocator sees a large variety of sizes, compiled kernels may specialize repeatedly, and peak memory fluctuates.

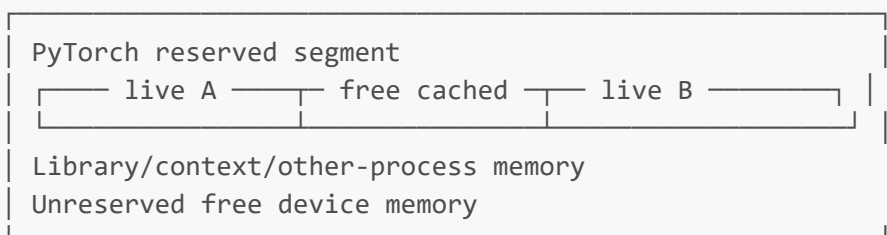
A practical design:

1. Bucket requests into a controlled set of sequence lengths.
2. Reuse preallocated input/output/KV-cache buffers where ownership is clear.
3. Run warm-up for expected buckets.
4. Track allocated, reserved, and peak memory per phase.
5. Avoid storing GPU tensors or graph-connected losses indefinitely in request logs.
6. Use an admission-control estimate that includes temporary workspace, not just model weights.

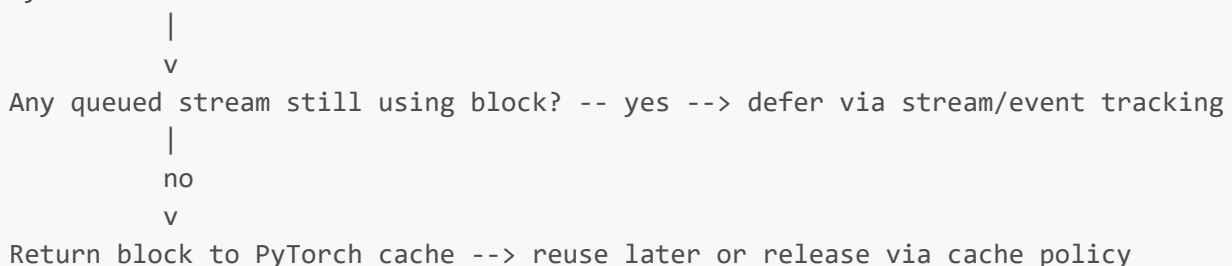
If an OOM occurs while reserved memory is high, the memory snapshot can reveal whether free blocks are fragmented or whether live allocations truly occupy the pool. The diagnosis determines whether to reduce batch size, stop retaining graphs, stabilize shapes, checkpoint activations, or adjust allocator configuration.

5. Diagrams / Mental Models

CUDA device capacity



Python refcount reaches zero



Metric	Includes	Excludes/limitations
allocated	Live allocator-managed tensor blocks	Free cached blocks, many external allocations
reserved	Allocated + cached allocator segments	Some context/library/custom allocations
peak allocated	High-water mark of allocated accounting	Does not by itself explain owners
driver/process tool	Broad process-visible device consumption	Does not identify individual tensors/blocks

6. Common Interview Questions

1. **Why does PyTorch cache CUDA memory?** To avoid repeated expensive/synchronizing driver allocations and frees. Expected: fast reuse in iterative workloads. Mistake: calling all retained memory a leak.
2. **What is the difference between allocated and reserved memory?** Allocated is used by live allocator-managed tensors; reserved includes cached free space held by the allocator. Expected: explain why the gap is normal. Mistake: using the terms interchangeably.
3. **Why can `nvidia-smi` stay high after `del`?** Freed blocks may remain reserved in the cache, and context/library allocations remain. Expected: deletion only removes a reference; garbage collection and asynchronous use also matter. Mistake: promising immediate driver release.
4. **What does `empty_cache()` do?** It releases eligible unused cached blocks/segments to CUDA for other consumers. It cannot free live tensors. Expected: separate cached-free from live memory. Mistake: treating it as a way to delete tensors or fix retained graphs.
5. **Should `empty_cache()` be called every iteration?** Usually no; it defeats reuse and can increase allocation overhead. Expected: use selectively at phase boundaries if needed. Mistake: recommending it as routine optimization.
6. **How can OOM occur when some memory is free?** No sufficiently suitable block may exist due to fragmentation, or unreserved capacity may be below the large request/temporary peak. Expected: sum of free bytes is not enough. Mistake: claiming CUDA can always combine arbitrary live-separated regions.
7. **What causes training peak memory?** Parameters, saved activations, gradients, optimizer states, temporaries/workspaces, and overlapping lifetimes. Expected: backward can combine categories. Mistake: counting only parameters.
8. **How do you diagnose a memory leak?** Track allocated/reserved/peaks by iteration, inspect object/graph retention and allocator snapshots, and minimize a reproducer. Expected: distinguish monotonic live allocation from stable reservation. Mistake: looking only at `nvidia-smi`.
9. **Why can storing loss in a list grow memory?** A tensor loss can retain its autograd graph and saved tensors. Store `loss.item()` for a scalar log when synchronization is acceptable, or an appropriately detached/CPU value. Expected: connect references to graph lifetime. Mistake: believing a scalar-shaped CUDA tensor has no graph.
10. **How do streams affect freeing?** A block cannot be reused until queued uses complete; cross-stream use may need `record_stream` or explicit ownership synchronization. Expected: distinguish host-object and queued-device lifetimes. Mistake: equating Python deletion with device completion.

11. **Why do variable shapes hurt allocation behavior?** They create varied block sizes, reduce reuse, change peaks, and can contribute to fragmentation. Expected: batching/bucketing/preallocation as remedies. Mistake: assuming the allocator always finds a perfect fit.
12. **Does mixed precision halve total training memory?** Not necessarily. Some states/accumulators remain FP32, activations vary, and temporary/workspace memory persists. Expected: category-by-category accounting. Mistake: multiplying the entire process footprint by 0.5.
13. **What is the safest first response to OOM?** Identify live owners and peak phase; then reduce unnecessary lifetimes/workload or use checkpointing/bucketing. Expected: diagnose before choosing a remedy. Mistake: immediately changing obscure allocator settings.

7. Deep-Dive Questions

1. **Why can freeing memory require event tracking rather than a device synchronize?** Events allow the allocator to learn when a particular stream has passed the last use without stalling the host or unrelated device work. This preserves asynchrony and safe reuse.
2. **What is internal versus external fragmentation here?** Internal fragmentation is unused space inside a block allocated due to rounding; external fragmentation is free space divided among blocks/segments so a large request cannot be served. Allocator policies trade one against the other.
3. **How can compilation/fusion change peak memory?** Fusion can eliminate intermediates and lower traffic, but compiler workspaces, persistent buffers, scheduling choices, and simultaneous lifetimes can sometimes increase particular peaks. Measure rather than assume.
4. **Why is a memory snapshot more useful than one aggregate number?** It can reveal allocation sizes, lifetimes, call sites, inactive splits, and the operation near the peak, separating fragmentation from retained live tensors.
5. **How can preallocation help and hurt?** It stabilizes addresses, avoids repeated allocation, and supports CUDA Graph replay, but can reserve worst-case capacity, complicate ownership, and waste memory for smaller requests.

8. Comparison Tables

Live allocation	Cached free block	Unreserved device memory
Owned by active tensor/workspace	Owned by allocator, reusable	Available to driver/other allocators
Counted allocated and reserved	Counted reserved, not allocated	Counted by neither PyTorch allocator metric
Cannot be released by <code>empty_cache()</code>	May be released when eligible	No PyTorch action needed
Real graph/tensor retention	Normal caching	
Allocated memory grows with iterations	Allocated stabilizes; reserved may stay high	
Live references/graphs remain	Blocks are free inside allocator	
<code>empty_cache()</code> cannot fix live owners	<code>empty_cache()</code> can expose free blocks to others	
Fix ownership/lifetime	Usually leave cache for performance	

9. Common Mistakes

- Treating `nvidia-smi` usage as live PyTorch tensor bytes.
- Calling `empty_cache()` in every iteration.
- Assuming `del` synchronously frees device memory to the driver.
- Ignoring Python references held in lists, closures, logs, or metrics.
- Saving graph-connected outputs for later inspection.
- Measuring only steady-state memory and missing transient peaks.
- Ignoring library workspaces and non-PyTorch CUDA allocations.
- Assuming total free bytes imply one large allocation can succeed.
- Forgetting cross-stream allocator lifetime.
- Applying allocator tuning before eliminating unnecessary tensor lifetimes.
- Hard-coding implementation-specific pool thresholds in an interview answer.

10. Edge Cases / Special Cases

- Python cycles may delay object destruction until garbage collection.
- Views can keep a much larger base storage alive.
- Asynchronous kernels can delay safe reuse after the last Python reference disappears.
- `empty_cache()` may synchronize or add future allocation cost depending on backend/state; it is not performance-neutral.
- CUDA Graph capture often requires stable addresses and special allocator pools/lifetimes.
- Distributed libraries, custom CUDA extensions, and external frameworks can allocate outside PyTorch's accounting.
- OOM error reporting can occur at a later synchronization because CUDA errors are asynchronous.
- Memory statistics are device-specific; reset peak counters deliberately when measuring phases.
- Deterministic or autotuned kernels may use different workspace amounts.
- Exact caching allocator backend, rounding, split policy, and configuration are version-dependent.

11. How to Explain in Interview

"PyTorch's CUDA caching allocator reserves device segments and reuses blocks so training does not repeatedly pay driver allocation and synchronization costs. `memory_allocated` represents live allocator-managed tensors, while `memory_reserved` also includes cached free blocks, so high `nvidia-smi` usage is not automatically a leak. For OOM I inspect live lifetimes, peaks, fragmentation, workspaces, and stream use; `empty_cache()` frees only unused cache and is not a substitute for fixing retained tensors."

12. Quick Revision Notes

- Cache exists for speed and asynchrony.
- Allocated = live blocks; reserved = allocator-held segments.
- Driver-visible memory includes more than PyTorch tensors.
- Deletion makes memory eligible; safe reuse may wait for streams.
- `empty_cache()` cannot free live tensors.
- Irregular sizes can hurt reuse and fragment pools.
- Graph retention is different from stable caching.
- Peak lifetime overlap matters more than final memory.
- Inspect snapshots/stats before tuning.

- Trap: a tiny view can retain huge storage.

13. Practice Tasks

1. Allocate/delete CUDA tensors and chart allocated versus reserved memory.
2. Create a real graph-retention bug by appending losses, then fix it and compare curves.
3. Generate alternating allocation sizes and inspect `memory_summary()`.
4. Measure peak memory for training, inference, mixed precision, and checkpointing.
5. Hold a small slice of a large tensor and verify that base storage remains live.
6. Run a cross-stream temporary example and explain allocator-safe lifetime.
7. Compare variable sequence lengths with bucketed lengths.
8. Capture a memory snapshot near OOM and classify live blocks, inactive splits, and external memory.

14. Final Cheat Sheet

Item	Must remember
Core definition	Segment/block cache over CUDA allocations for fast reuse.
Metrics	Allocated = live; reserved = live + cached allocator space.
OOM causes	Capacity, lifetime, transient peak, fragmentation, external use.
Most asked	<code>empty_cache</code> , <code>nvidia-smi</code> , leak diagnosis, stream safety.
Common comparison	Live retention versus normal cached reservation.
Interview trap	Free cached memory is not necessarily returned to the driver.
One-line answer	"The allocator keeps freed blocks for speed; diagnose live ownership and peaks before blaming the cache."

Kernel Launches

1. Overview

A **kernel launch** is the act of submitting a GPU function and its execution configuration to a CUDA stream. In PyTorch, one high-level operator may launch one kernel, several kernels, a vendor-library routine that launches kernels, or no new kernel if it is metadata-only.

Kernel launches matter because each submission has CPU/framework/driver overhead and because the GPU needs enough parallel work to be efficient. Thousands of tiny operators can be launch-bound even when their arithmetic is trivial. Launches are used for elementwise operations, reductions, matrix multiplications, copies, initialization, backward formulas, and communication support.

Interviewers ask about kernel launches to connect PyTorch code with CUDA grids/blocks, asynchronous execution, launch overhead, occupancy, error reporting, profiling, and why vectorized Python-looking code can still create many separate GPU passes.

2. Core Idea

Think of a launch as sending a work order to a huge factory. Filling out and delivering the order costs time regardless of whether the job performs ten additions or ten billion. Large jobs amortize the paperwork; many tiny jobs spend most of their time on paperwork and setup.

```
x = torch.randn(1_000_000, device="cuda")
y = torch.relu(x + 1) * 2
```

In ordinary eager execution, this expression may involve separate operator/kernel work for addition, ReLU, and multiplication, plus intermediate allocations. Exact launches depend on PyTorch version, dispatch path, and compilation. Conceptually:

```
CPU/Python -> dispatcher -> CUDA implementation -> enqueue kernel in stream
GPU stream: [add kernel] -> [relu kernel] -> [multiply kernel]
```

Step by step:

1. Python calls a PyTorch operator.
2. The dispatcher chooses an implementation based on device, dtype, layout, and other keys.
3. The CUDA implementation selects/configures a kernel or library call.
4. Arguments and grid/block configuration are submitted to the current stream.
5. The host call commonly returns before GPU completion.
6. The GPU schedules thread blocks onto SMs as resources become available.
7. Execution faults may surface only at a later synchronization point.

3. Important Subtopics

PyTorch dispatch path

An operator schema/API is resolved through PyTorch dispatch to a backend implementation. Autograd, autocast, functionalization, compilation, or custom backends can participate around the core kernel.

Why it matters: “one Python line” is not a reliable kernel count, and CPU overhead can occur before the launch.

Example: `torch.nn.functional.linear` can invoke matrix multiplication and bias logic through optimized library paths.

Interview angle: distinguish operator, kernel, and library call.

Grid, block, thread, and SM

A CUDA kernel launch defines a grid of thread blocks. Threads in a block can cooperate through shared memory and barriers; blocks are scheduled onto streaming multiprocessors (SMs).

Why it matters: block size, registers, shared memory, and work per thread influence occupancy and throughput.

Example: an elementwise kernel often maps one or multiple elements to each thread using a grid-stride loop.

Interview angle: PyTorch users may not choose launch dimensions directly, but custom kernel authors must.

Launch overhead and small operations

Every launch has nonzero host submission and device scheduling overhead. Framework dispatch and Python overhead can add more.

Why it matters: tiny tensors/short kernels can be latency-bound, and a fast GPU cannot accelerate overhead it has not yet received.

Example: 1000 scalar-like CUDA operations are far slower than one vector operation over 1000 elements.

Interview angle: batch, vectorize, fuse, compile, or capture repeated static workloads.

Asynchronous error reporting

Invalid launch configuration can be detected around submission, but faults such as illegal memory access occur during device execution and may surface later.

Why it matters: the Python stack where the error appears may not be the source.

Example: diagnostic synchronous-launch modes can localize errors but severely alter timing and should not be normal production settings.

Interview angle: check immediate launch errors and a suitable synchronization boundary in custom CUDA debugging.

Occupancy versus utilization

Occupancy is the ratio of active warps to the hardware maximum, constrained by registers, shared memory, blocks, and architectural limits. High occupancy helps hide latency but is not identical to high performance.

Why it matters: a kernel can be bandwidth-bound at moderate occupancy or waste work at high occupancy.

Interview angle: occupancy is a means for latency hiding, not the optimization goal itself.

Memory-bound and compute-bound kernels

Arithmetic intensity compares operations to bytes transferred. Elementwise kernels are often memory-bound; matrix multiplications can be compute-bound with good reuse.

Why it matters: optimization should target the bottleneck. Fusion helps memory-bound chains by reducing intermediate traffic; it may not improve an already optimal GEMM core.

Interview angle: use roofline-style reasoning.

Libraries, autotuning, and workspaces

PyTorch uses libraries such as cuBLAS/cuDNN and generated/native kernels. A library call may select among algorithms using shape, dtype, determinism, and workspace constraints.

Why it matters: first-call initialization/autotuning can differ from steady state, and one operator may launch multiple internal kernels.

Interview angle: warm up before benchmarking and report shape/dtype/device/settings.

CUDA Graph replay

CUDA Graphs record a sequence of launches and replay it with much lower repeated CPU submission overhead, given static-enough shapes, addresses, and control flow.

Why it matters: graph replay is valuable for launch-bound stable workloads.

Interview angle: capture reduces launch overhead; it does not fuse all kernels into one.

4. Real-World Example

An autoregressive language model generates one token at a time. Each step contains many small normalizations, projections, elementwise updates, and cache operations. Batch size may be small, so individual kernels are short and CPU launch latency becomes visible between them.

Optimizations target different layers:

- Batch requests so each launch has more work.
- Use fused attention/normalization/optimizer kernels to reduce launch count and memory traffic.
- Compile stable graph regions to reduce Python/dispatcher overhead and generate fused kernels.
- Use CUDA Graph replay when shapes and addresses can be stabilized.
- Avoid `.item()` and host decisions that introduce gaps/synchronization.

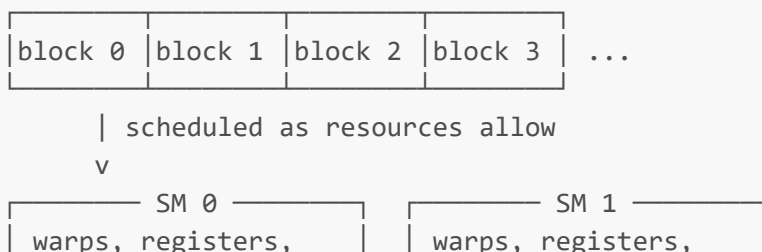
A GPU timeline shows whether gaps between kernels come from CPU submission, synchronization, data transfer, compilation, or dependencies. Kernel duration alone cannot diagnose launch-bound behavior.

5. Diagrams / Mental Models

```
Host timeline: dispatch A | launch A | dispatch B | launch B | ...
GPU timeline:      [A]          gap      [B]
```

If kernels are tiny, host submission may fail to keep the GPU busy.

Grid



shared memory

shared memory

Symptom	Likely direction to investigate
Tiny kernels with GPU gaps	Launch/CPU overhead, synchronization
Long kernel, low memory and compute throughput	Divergence, dependencies, poor access, insufficient parallelism
High bandwidth, low arithmetic intensity	Memory-bound; reduce traffic/fuse
High math throughput	Compute-bound; algorithm/precision/hardware units
Long first iteration only	Initialization, compilation, autotuning, allocation

6. Common Interview Questions

1. **What is a CUDA kernel launch?** Submission of a device function, arguments, and grid/block configuration to a stream. Expected: usually asynchronous to the host. Mistake: saying the call waits for every GPU thread.
2. **Is one PyTorch operation one kernel?** Not necessarily. It may be metadata-only, one kernel, multiple kernels, or a library routine with internal launches. Expected: distinguish API operator, backend routine, and kernel. Mistake: counting Python lines as kernels.
3. **Why are many small GPU operations slow?** Launch, dispatch, allocation, and synchronization overhead are poorly amortized, and small grids underuse the GPU. Expected: mention both host overhead and insufficient parallel work. Mistake: assuming GPU arithmetic itself is slow.
4. **What is the difference between a grid and a block?** A grid contains blocks; threads within a block share cooperative resources and synchronization, while blocks are independently scheduled. Expected: explain scheduling and cooperation scope. Mistake: claiming threads in arbitrary blocks can use a normal block barrier.
5. **What is occupancy?** Active warps relative to the SM's supported maximum, limited by resources. Expected: latency-hiding metric, not performance itself. Mistake: optimizing occupancy to 100% at any cost.
6. **What determines block size?** Work mapping, warp size, register/shared-memory use, occupancy, memory access, and algorithmic cooperation. Expected: benchmark/occupancy analysis. Mistake: saying 1024 threads is always best.
7. **Why is launch timing with `time.time()` wrong?** The host measures asynchronous submission unless synchronized. Expected: CUDA events or defined synchronization. Mistake: synchronizing after every op in production.
8. **Why can an illegal memory access appear at a later line?** Execution is asynchronous, so the fault is reported at a later CUDA API/synchronization. Expected: use diagnostic synchronization to localize. Mistake: assuming the reporting line caused it.
9. **What is a memory-bound kernel?** Performance is limited mainly by memory traffic/bandwidth rather than arithmetic throughput. Expected: low arithmetic intensity. Mistake: equating long runtime with compute-bound.
10. **How can launch overhead be reduced?** Fuse/vectorize/batch work, compile graph regions, use persistent approaches where appropriate, or replay CUDA Graphs. Expected: choose a technique based

on workload stability and semantics. Mistake: increasing stream count as the only answer.

11. **Does CUDA Graph capture fuse kernels?** No. It captures the launch/dependency graph for lower-overhead replay; kernels generally remain distinct. Expected: separate submission optimization from code-generation optimization. Mistake: confusing graph replay with compiler fusion.
12. **What should a custom PyTorch CUDA kernel do about streams?** Launch on the current PyTorch CUDA stream and obey input/output lifetime and error conventions. Expected: integrate with current-stream ordering. Mistake: hard-code stream 0.
13. **Why can the first launch be much slower?** CUDA context/library initialization, allocator growth, JIT compilation, module loading, or autotuning. Expected: warm-up and report steady-state separately. Mistake: discarding all first-call cost when startup latency matters to the product.
14. **Can two kernels execute concurrently?** Yes across streams if dependencies and resources allow, but not guaranteed. Expected: discuss stream independence and resource availability. Mistake: assuming concurrent kernels always increase throughput.

7. Deep-Dive Questions

1. **How do registers affect launch performance?** Registers are allocated per thread/warp. High register use can reduce resident blocks/warps; spilling adds local-memory traffic. Reducing registers can improve occupancy but may increase instructions/spills, so tune empirically.
2. **Why can a grid be too small?** If it has fewer runnable blocks/warps than the GPU can host, many SMs sit idle. Increasing per-thread work does not replace enough independent parallel work for latency hiding.
3. **What is a persistent kernel?** A longer-running kernel keeps workers resident and processes multiple tasks from device-side queues, amortizing launches. It can reduce latency but complicates scheduling, fairness, occupancy, and integration.
4. **How does fusion affect launch configuration?** The fused kernel must accommodate combined resource requirements. Extra registers or shared memory can reduce occupancy, so fewer launches and less traffic do not guarantee a faster kernel.
5. **What timeline evidence distinguishes launch-bound from compute-bound execution?** Launch-bound traces show many short kernels and gaps correlated with CPU submission; compute-bound traces show long kernels with high compute-unit activity and little idle gap. CPU and GPU traces must be viewed together.

8. Comparison Tables

Operator	Kernel	CUDA Graph
User/framework semantic action	One GPU program execution	Captured dependency/launch sequence
May dispatch to zero, one, or many kernels	Has grid/block/resource configuration	Replays multiple operations cheaply
Example: <code>softmax</code>	Reduction/normalization device routine	Static inference iteration replay

Launch-bound workload	Memory-bound workload	Compute-bound workload
Time dominated by submission/gaps	Time dominated by data movement	Time dominated by arithmetic units
Many tiny kernels	Low operations per byte	High arithmetic intensity
Fuse/capture/batch	Reduce bytes, improve locality/fuse	Better algorithm/precision/math units
GPU often idle between work	Bandwidth near limit	Compute throughput near limit

9. Common Mistakes

- Equating one Python statement, one operator, and one kernel.
- Timing only asynchronous submission.
- Assuming every GPU slowdown means low occupancy.
- Maximizing occupancy while causing extra spills or work.
- Ignoring CPU dispatcher/launch gaps in a GPU trace.
- Assuming more streams remove launch overhead.
- Treating CUDA Graph replay as kernel fusion.
- Benchmarking without warm-up or with hidden synchronization.
- Hard-coding custom extensions to the default stream.
- Ignoring dtype/layout/shape in kernel selection.
- Using a tiny test shape to judge a production kernel.

10. Edge Cases / Special Cases

- Metadata-only operations such as many views can launch no kernel.
- Lazy initialization and just-in-time compilation make cold-start launch cost atypical.
- Empty tensors may short-circuit without a meaningful grid.
- Dynamic shapes can cause recompilation or different launch configurations.
- Vendor library calls can enqueue auxiliary kernels and use temporary workspaces.
- Determinism requirements can select different algorithms/kernel sequences.
- Kernel failures are “sticky” enough that later CUDA calls may continue reporting an error until handled/process reset as appropriate.
- Cooperative launches and very large dynamic shared memory have extra constraints.
- Profilers add overhead; use representative runs and understand their collection mode.
- Windows GPU watchdog/display scheduling can affect very long kernels on display-attached devices.

11. How to Explain in Interview

“A PyTorch CUDA operator dispatches to a backend that enqueues one or more kernels on the current stream. Each launch specifies a grid of blocks and returns to the CPU before completion in the normal case. Performance depends on amortizing dispatch/launch overhead, exposing enough blocks, and matching resource use to whether the kernel is launch-, memory-, or compute-bound. I use a CPU/GPU timeline and CUDA events rather than counting Python lines or wall-clocking asynchronous calls.”

12. Quick Revision Notes

- Operator count is not kernel count.
- Launch = function + args + grid/block + stream submission.
- Normal launch is host-asynchronous.
- Tiny kernels can be launch-bound and underfill the GPU.
- Occupancy helps hide latency but is not the goal.
- Arithmetic intensity separates memory/compute direction.
- Warm up libraries, allocators, and compilers for steady-state tests.
- Events measure device elapsed time.
- CUDA Graph replay reduces submission overhead, not kernel count by fusion.
- Trap: errors can surface after the faulty launch.

13. Practice Tasks

1. Profile `relu(x+1)*2` in eager and compiled modes; count actual kernels.
2. Sweep tensor sizes and identify when an elementwise op becomes launch- versus bandwidth-dominated.
3. Compare 1000 small adds with one batched add.
4. Use CUDA events and host timers to show asynchronous timing differences.
5. Inspect a matrix multiplication's achieved compute and an elementwise kernel's memory throughput.
6. Write pseudocode for grid-stride indexing and choose a block size experimentally in a custom kernel framework.
7. Capture a repeated static step with CUDA Graphs and inspect whether kernels remain separate.
8. Introduce a custom out-of-bounds access in a safe toy environment and practice localizing asynchronous failure.

14. Final Cheat Sheet

Item	Must remember
Core definition	Submit configured GPU work to a CUDA stream.
Main property	Host-asynchronous, same-stream ordered.
Performance question	Launch-, memory-, or compute-bound?
Most asked	Grid/block, occupancy, timing, small-kernel overhead.
Common comparison	Semantic operator versus physical kernel launch.
Interview trap	One Python line may launch zero, one, or many kernels.
One-line answer	"A launch queues a configured grid; good GPU performance amortizes that queueing and keeps hardware busy with the right bottleneck strategy."

Operator Fusion

1. Overview

Operator fusion combines multiple logical tensor operations into fewer physical kernels or library operations. Instead of writing an intermediate tensor to global memory and launching the next kernel to read it, a fused kernel can keep intermediate values in registers/shared memory and produce the final result directly.

Fusion matters because many neural-network operations are memory-bound and eager execution can create numerous short kernels and temporary allocations. Fusion can reduce kernel launches, global-memory traffic, allocator pressure, and sometimes synchronization. It is used in activations, bias/normalization patterns, attention, optimizers, pointwise chains, and compiler-generated graphs.

Interviewers ask about fusion to see whether candidates understand what it saves, when it helps, how compilers preserve semantics, and why a larger fused kernel can sometimes be slower due to register pressure, reduced occupancy, recomputation, or lost library optimizations.

2. Core Idea

Suppose eager execution computes:

```
y = torch.relu(x + bias) * scale
```

An unfused conceptual path is:

```
kernel 1: read x,bias -> write temp1
kernel 2: read temp1   -> write temp2 (ReLU)
kernel 3: read temp2,scale -> write y
```

A fused path is:

```
one kernel per element:
  v = x[i] + bias[broadcast_index(i)]
  v = max(v, 0)
  y[i] = v * scale
```

Step by step:

1. Read each necessary input.
2. Compute several intermediate expressions inside one thread/program.
3. Keep temporary values in registers when possible.
4. Write only the final output.
5. Avoid intermediate allocations and extra launch submissions.

The mathematical operators remain conceptually separate in the model; fusion changes physical execution. A compiler must preserve dtype promotion, broadcasting, aliasing, randomness, exceptions, and autograd semantics within allowed numerical tolerances.

3. Important Subtopics

Vertical/producer-consumer fusion

A producer and its consumer are combined so the intermediate stays local.

Why it matters: eliminates write/read of a temporary and one or more launches.

Example: `add -> sigmoid -> multiply` in an activation gate.

Interview angle: strongest when the intermediate has one consumer and operations share compatible iteration spaces.

Horizontal fusion

Independent or sibling operations with compatible inputs/iteration domains are computed together.

Why it matters: reduces launch count and can reuse an input load, but combined outputs/resource usage may grow.

Example: compute two pointwise projections/statistics from the same activation in one kernel where a compiler/library supports it.

Interview angle: distinguish it from producer-consumer fusion.

Epilogue fusion

High-performance GEMM/convolution kernels can fuse work such as bias, scaling, activation, or residual handling into their output epilogue.

Why it matters: the expensive core remains a tuned library kernel while the output is transformed before an extra round trip.

Example: matrix multiplication plus bias plus GELU.

Interview angle: fusing arbitrary logic into GEMM may lose a best-in-class library kernel; epilogue support is a valuable compromise.

Reduction fusion

Reductions have different iteration/communication patterns from pointwise operations. Producers can often fuse into a reduction, and consumers may sometimes fuse after it, but synchronization and intermediate reduction results constrain the schedule.

Why it matters: layer normalization and softmax require reductions plus normalization/pointwise work and benefit from specialized fused kernels.

Example: compute max, exponentials, sum, and normalization for a row-wise softmax with carefully staged passes/online algorithms.

Interview angle: a global synchronization boundary cannot simply be erased inside an ordinary kernel.

Eager fusion, library fusion, and compiler fusion

Fusion may come from a hand-written fused operator, a vendor/library epilogue, or compilation such as `torch.compile` generating/choosing fused kernels for captured graph regions.

Why it matters: hand-written kernels can be excellent but narrow; compiler fusion covers more patterns but must specialize and guard dynamic behavior.

Interview angle: exact compiler components evolve; focus on graph capture, legality, scheduling, code generation, and guards.

Fusion legality

Aliasing/in-place mutation, observable ordering, data-dependent control flow, unsupported operations, device changes, dynamic shapes, randomness, and layout constraints can block or split fusion.

Why it matters: changing operation order can change visible behavior or numeric results.

Example: a graph break around a Python scalar extraction prevents one fused region from spanning that boundary.

Interview angle: fusion is not a text substitution of adjacent Python lines.

Backward fusion

Forward and backward graphs can each be optimized. A fused forward may have a hand-written fused backward or the compiler may jointly/recomputed schedule derivative expressions.

Why it matters: total training speed and saved activation memory matter more than forward-only speed.

Example: fused dropout-add-layernorm requires correct RNG state/mask handling and parameter-gradient reductions.

Interview angle: ask what must be saved for backward and whether recomputation is cheaper.

Costs of over-fusion

Larger kernels can use more registers/shared memory, lower occupancy, increase compile time/code size, create redundant recomputation, or prevent reuse of an intermediate by multiple consumers.

Why it matters: the best fusion boundary is a performance decision constrained by semantics.

Interview angle: “fuse everything” is not a mature answer.

4. Real-World Example

Transformer layer normalization illustrates why fusion is valuable. An unfused implementation might launch kernels for mean, variance, subtract, reciprocal square root, normalize, multiply weight, and add bias. It also creates and rereads intermediates.

A specialized fused kernel can:

1. Load a row of activations.
2. Compute mean/variance using warp/block reductions.
3. Normalize values.

4. Apply learned scale and bias.
5. Write one final activation tensor.

For training, it must save or recompute appropriate statistics and provide a backward that reduces gradients for weight and bias. Performance depends on hidden size, dtype, alignment, memory layout, and kernel resources.

Likewise, fused attention avoids materializing the full score/probability matrices in some algorithms by processing tiles and maintaining online softmax statistics. This is deeper than simply placing three Python operators into one elemental loop; the algorithm and memory schedule are redesigned.

5. Diagrams / Mental Models

Unfused global-memory traffic

```
inputs -> [add] -> temp A -> [relu] -> temp B -> [mul] -> output
           W A              R W              R W
```

Fused

```
inputs -> [ add + relu + mul in registers ] -> output
           read inputs once-ish, write final once
```

Candidate fusion region

```
|
+-- semantics legal? ---- no --> split
|
+-- compatible schedule/layout? -- no --> split or transform
|
+-- resource/performance profitable? -- no --> keep separate
|
+-- yes --> generate/select fused kernel and benchmark
```

Benefit	Source
Fewer launches	Multiple logical ops submitted as fewer kernels
Less global traffic	Intermediates stay in registers/shared memory or are recomputed
Fewer allocations	No storage for eliminated intermediates
Better locality	Producer value consumed immediately
Possible downside	More resources, code, compile time, or less specialized kernels

6. Common Interview Questions

1. **What is operator fusion?** Combining multiple logical operations into fewer physical kernels/operations while preserving semantics. Expected: fewer launches and intermediate memory traffic. Mistake: calling

it merely Python function inlining.

2. **Why does fusion help GPUs?** It amortizes launch overhead and keeps intermediates on-chip, reducing global reads/writes and allocations. Expected: especially useful for memory-bound pointwise chains. Mistake: saying it reduces the mathematical operation count in every case.
3. **Does fusion always improve performance?** No. Register/shared-memory pressure, lower occupancy, compilation overhead, recomputation, or losing tuned library paths can hurt. Expected: evaluate resources and end-to-end cost. Mistake: "one kernel is always fastest."
4. **What is vertical fusion?** Combining a producer with its consumer. Expected: eliminate the intermediate. Mistake: confusing it with batching unrelated inputs.
5. **What is horizontal fusion?** Combining compatible sibling/independent computations, often sharing an input or iteration domain. Expected: launch/input reuse benefits. Mistake: requiring a producer-consumer edge.
6. **What is epilogue fusion?** Applying bias/activation/scaling/residual-style work as part of a GEMM/convolution output stage. Expected: preserve optimized core kernel. Mistake: saying it fuses two arbitrary GEMMs.
7. **Why are reductions harder to fuse?** They require cross-thread aggregation and synchronization, and consumers may need a completed reduced value. Expected: block/warp/global boundaries. Mistake: treating softmax as a simple pointwise chain.
8. **How does `torch.compile` help?** It captures graph regions, applies transformations/fusion and code generation or kernel selection, guarded by assumptions. Expected: graph breaks/dynamic specialization can limit gains. Mistake: promising one kernel for the whole model.
9. **What can prevent fusion?** Aliasing/mutation, unsupported ops, device/layout changes, graph breaks, incompatible iteration spaces, randomness constraints, or profitability decisions. Expected: include both legality and profitability barriers. Mistake: blaming only dynamic shapes.
10. **How does fusion affect autograd?** Forward and backward must remain differentiable/correct; fusion may change saved intermediates, recomputation, and backward kernel grouping. Expected: discuss saved state and backward execution. Mistake: analyzing only inference.
11. **How do you verify fusion?** Inspect compiler output/logs where appropriate and use a profiler to count kernels, allocations, traffic, and total latency. Expected: verify correctness and warm-up/compile cost separately. Mistake: infer fusion from one Python line.
12. **Can fusion improve memory capacity?** Yes, eliminating intermediates can lower live/peak allocated memory. Expected: not guaranteed because workspaces or longer lifetimes may offset it. Mistake: claiming all fusion halves memory.
13. **What is the difference between fusion and CUDA Graphs?** Fusion changes kernel boundaries/data movement; CUDA Graphs replay a sequence with lower submission overhead while usually retaining kernel boundaries. Expected: contrast memory traffic/kernel count with submission overhead. Mistake: using the terms interchangeably.
14. **Why might fusing two compute-heavy operators be unattractive?** Each may already use highly optimized kernels/resources, and combining can reduce scheduling flexibility or be algorithmically incompatible. Expected: target boundaries around memory-bound epilogues instead. Mistake: optimizing launch count alone.

7. Deep-Dive Questions

1. **How does a compiler decide whether fusion is legal?** It analyzes data dependencies, mutation/aliasing, observable order, device/dtype/layout, shape guards, and operator semantics. Only transformations that preserve permitted numerical/side-effect behavior are candidates.

2. **How does it decide whether fusion is profitable?** A cost model or autotuning estimates launch savings, bytes removed, recomputation, resource usage, shape, and available implementations. Actual systems may benchmark candidate schedules.
3. **What is recompute fusion?** Instead of materializing a cheap producer used in multiple places, a fused consumer recomputes it locally. This trades extra arithmetic for less global memory; it is attractive when arithmetic is cheap and memory traffic dominates.
4. **Why can fused kernels increase numerical differences?** Reassociation, changed reduction order, lower-precision intermediates, or fused multiply-add can alter rounding. Results should satisfy the framework's allowed numerical contract, but bitwise identity may not hold unless required.
5. **How does fused attention avoid a quadratic materialized matrix?** Tiled algorithms stream blocks of keys/values, update numerically stable online softmax statistics, and accumulate output tiles without storing the entire attention matrix. It is algorithmic fusion plus tiling, not only launch coalescing.

8. Comparison Tables

Unfused execution	Fused execution
More launches	Fewer launches
Materialized intermediates	Intermediates may stay on chip/recompute
Simple modular kernels	More complex combined schedule
Lower per-kernel resources	Potentially higher registers/shared memory
Easy reuse of intermediates	May duplicate work for multiple consumers
Lower compile complexity	More compilation/autotuning cost

Operator fusion	CUDA Graph capture
Changes physical kernel boundaries	Records/replays launch sequence
Reduces launches and/or memory traffic	Reduces CPU submission overhead
May generate new code	Reuses captured kernels/addresses
Constrained by fusion legality/profitability	Constrained by capture safety/static state
Can benefit one execution after compilation	Best for repeated stable execution

Hand-written fused op	Compiler-generated fusion
Expert-optimized for a pattern	Broader automatic coverage
Predictable supported contract	Depends on captured graph and guards
Maintenance burden and narrow shapes/dtypes	Compile overhead and possible graph breaks
Excellent for critical stable patterns	Excellent for varied compositional pointwise graphs

9. Common Mistakes

- Saying fusion is only fewer function calls in Python.
- Assuming adjacent source lines are automatically fusible.

- Claiming one kernel is always faster than several.
- Ignoring register pressure, occupancy, and compile time.
- Ignoring backward and saved-tensor semantics.
- Comparing compiled first-run time with warmed eager time.
- Treating CUDA Graph capture as fusion.
- Expecting an entire neural network to become one kernel.
- Fusing around a GEMM in a way that loses an optimized library implementation.
- Ignoring numerical differences from reordered reductions/operations.
- Looking only at kernel count instead of end-to-end latency and memory.

10. Edge Cases / Special Cases

- Metadata-only views may be folded into indexing without a standalone kernel.
- Aliasing and in-place mutation can require graph functionalization or block fusion.
- Random operators require correct seed/offset handling; fusion must not change the random-number contract unexpectedly.
- Dynamic shapes may be supported through guards/general kernels or may cause specialization/recompilation.
- A shared intermediate with multiple consumers creates a materialize-versus-recompute tradeoff.
- Reduction dimensions, non-contiguous strides, and broadcasting can make schedules incompatible.
- Sparse, quantized, custom, or data-dependent operators may lack fusion support.
- Compilation caches consume host/device resources and cold-start time matters for short-lived programs.
- Deterministic settings can limit legal algorithms/reduction orders.
- Fused kernels can reduce allocator activity yet extend the lifetime of some inputs.

11. How to Explain in Interview

“Operator fusion combines compatible logical operations into fewer GPU kernels. Its main wins are fewer launches, fewer intermediate allocations, and less global-memory traffic because producer values can stay in registers or shared memory. It is especially effective for memory-bound pointwise and normalization patterns. Fusion must preserve aliasing, dtype, RNG, and autograd semantics, and over-fusion can increase register pressure or lose tuned library paths, so I verify it in a profiler and measure end-to-end.”

12. Quick Revision Notes

- Fusion changes physical execution, not model mathematics.
- Main savings: launches, temporary storage, global traffic.
- Vertical = producer-consumer; horizontal = compatible siblings.
- Epilogue fusion extends a tuned GEMM/convolution output stage.
- Reductions need special synchronization/tiling schedules.
- Compiler needs legality plus profitability.
- Backward can be fused too; saved tensors/recompute matter.
- Over-fusion can lower occupancy or increase compilation.
- CUDA Graphs reduce replay overhead but do not inherently fuse.
- Trap: fewer kernels is a metric, not the final objective.

13. Practice Tasks

1. Profile a pointwise chain in eager and `torch.compile` modes; compare launches, bytes, and latency.
2. Manually estimate global-memory traffic for three unfused pointwise kernels versus one fused kernel.
3. Compare a separate GEMM+bias+activation with an available epilogue-fused path.
4. Create a graph break using `.item()`/Python control flow, then restructure and inspect fusion regions.
5. Measure cold compilation time separately from warm execution and compute the break-even iteration count.
6. Study a fused layer-normalization implementation and identify reduction, saved state, and backward work.
7. Construct a multiple-consumer example and reason about materialization versus recomputation.
8. Compare operator fusion with CUDA Graph replay on the same launch-bound workload.
9. Validate fused and unfused gradients/numerics across dtypes and edge shapes.

14. Final Cheat Sheet

Item	Must remember
Core definition	Combine logical ops into fewer physical kernels/operations.
Main benefit	Fewer launches, intermediates, allocations, and global-memory passes.
Best target	Memory-bound compatible producer-consumer chains and epilogues.
Most asked	Why it helps, when it hurts, compile vs graph capture.
Common comparison	Fusion changes kernels; CUDA Graphs replay kernels cheaply.
Interview trap	Maximum fusion can lose performance through resources or bad boundaries.
One-line answer	"Fusion keeps intermediate values close to compute, but the best boundary is the one that preserves semantics and wins end-to-end."